Portfolio Assignment Assessment

Vilijam Cekov and Luiz Felipe De Souza Simao
BI Norwegian Business School
GRA4158
Marketing and the Analysis of Experiments and Quasi-experiments
May. 11, 2026

Portfolio Assignment Assessment

Contents

Portfolio task 2

For this exercise, we would like to present an experiment regarding the profitability of gachapon machine style mechanics (hereafter referred to as gacha) in video games. As a context for this research, we would like to raise the following scenario for evaluation. We have a free-to-play mobile game that offers players the ability to buy, using real money, currency within the video game. This currency is then used to obtain a highly desirable 'carrot' in the form of a player cosmetic (skin) through the gacha system. In this game, the player is shown the most motivating skin according to the developer's knowledge of their user base. The player is then presented with the opportunity to spend their currency for the chance (a roll) to obtain this skin (the stick) (Stankovic, 2021); to keep this mechanic distinct from gambling, a pity system is put in place, allowing the player to always be able to get the skin they want given that they roll enough times. To the player, the number of rolls that are missing for them to be able to win through pity occurs as a simple number on the initial screen where the player can choose to do the rolls.

1. Research question

Within this context, we would like to research whether *the visual salience of reward proximity causes a change in consumer pulling behaviour*. By physicalising the pity system and inducing a sense of momentum towards the carrot, we expect to produce an acceleration towards that goal by the players following the **goal-gradient hypothesis** (Liu, 2007).

2. Operationalisation and variables

To answer this question, we define an experiment designed to explicitly manipulate our focal independent variable: the player's psychological awareness of their proximity to a guaranteed reward (theoretical construct of visual salience of reward proximity). The specific treatment used to operationalise this variable is the implementation of a graphical progress bar showing "X pulls remaining until guaranteed rare item." By introducing this treatment, we directly manipulate the focal independent variable while keeping the underlying mathematical odds constant. Therefore, the progress bar should be nonintrusive and follow the game's established aesthetic so as not to break the heuristic and introduce additional elements of intrigue. Contrasting this group, we would have a control group that retains the existing pity system, but it is only subtly mentioned as a hidden count or plain number on the initial screen.

- **Focal Independent Variable vs. Treatment:** The focal independent variable is the visual salience of reward proximity. The treatment used to manipulate this is the "Visible Bar" condition, which is tested against the "Hidden Count" counterfactual control.

To run this experiment, we expect that our relevant dependent variables are (1) short-term pulling intensity, (2) monetization, and (3) retention and long-term satisfaction. We explicitly exclude long-term loyalty from our outcome measures; instead, we focus on satisfaction and retention, as satisfaction is a more proximal and significantly less noisy metric for evaluating the impact of this specific mechanic. We will operationalise these variables using the following specific KPIs:

1. **Pulling Intensity:** We will measure the frequency of engagement via **pulls per active day**, the **time-to-next-pull** (to capture momentum), and the **reach-pity rate** (the percentage of players who

complete the cycle to the guarantee).

2. **Monetization:** We will track transaction size and volume through **payer conversion rates**, **ARPU** (Average Revenue Per User), **ARPPU** (Average Revenue Per Paying User), and total **net revenue**.
3. **Retention:** To gauge the ”Rewarded Behaviour” effect, we will track **D1, D7, and D30 retention rates** along with the overall **churn rate** within the study window. Additionally, we will measure long-term satisfaction (e.g., via targeted in-game sentiment surveys or proxy engagement metrics) to evaluate how the visual system influences ongoing player sentiment over time.

3. Experimental design strategy

We believe that the best design strategy is a **Between-Subjects design** as our core experimental structure. This choice is specifically intended to isolate the treatment effect by assigning different subjects to the ’Visible Bar’ and ’Hidden Count’ conditions, thereby avoiding the carry-over and learning effects common in within-subject designs (see Section 4.1 for a detailed analysis of these internal validity threats). The experiment is operationalised as a Field Experiment employing a Randomised Controlled Trial (RCT) logic. Although a laboratory setting offers superior environmental control, we have prioritised impulsive financial behaviours in a real-world context (see Section 4.2 for the trade-offs between internal control and authenticity)

Because we can assume to have full control over the platform, we can ensure perfect randomised assignment through server-side feature-flagging mitigating cross-over. However, acknowledging that we cannot force player behaviour (as some may never visit the gacha screen), we adopt an **Intent-to-Treat (ITT)** framework to measure the practical business impact. We will further address potential **one-sided non-compliance** by analysing the **Average Treatment Effect on the Treated (ATET)** to isolate the impact on players who were actually exposed to the new visual cues.

4. Validity Analysis

For this experiment, we expect some threats to internal and external validity. Firstly, we expect **compound treatment** to affect internal validity if info accessibility varies. First, we expect compound treatment to occur, which affects internal validity. We mitigate this by ensuring both groups see the pity info on the same screen, varying only the *visual salience* (text vs. bar). Secondly, we expect **selection on returns** affecting external validity due to heterogeneous effects (Braun and Schwartz, 2025). ”Whales” (heavy spenders) might already track pity manually, while ”Light spenders” might be highly motivated by the bar. **To mitigate this, we will stratify our randomisation based on prior spending and engagement levels**, ensuring that high-value users are balanced across both groups. Third, we expect possible word-of-mouth interference. If players in the treatment group post screenshots of the ”new bar” on social media, players in the control group may become aware of it, violating the **Stable Unit Treatment Value Assumption (SUTVA)**. We will monitor social channels during the test window to assess the extent of this spillover.

4.1 Internal validity threats

To ensure the internal validity of our findings, we must carefully address several design threats. First, we considered the risk of anchoring and learning effects. An *alternative* within-subjects design, where a player sees the hidden count for one month and the visible bar for the next, would inevitably introduce carry-over effects. Once a player becomes cognisant of the pity system mechanics via the bar, it is impossible for them to ”un-learn” that behaviour when moving back to a control state. Therefore, our choice of a **between-subjects** design, where players are randomly assigned to either the ”Visible Bar” or ”Hidden Count” version of the app, serves as our primary mitigation strategy. This approach entirely avoids these carry-over effects, providing a cleaner estimate of the **Average Treatment Effect (ATE)**.

Another significant threat is the potential for compound treatment. The introduction of a ”progress bar” might unintentionally vary other aspects of the user experience, most notably information accessibility. If the new visual bar makes the pity count inherently easier to find than the existing text format, we risk confounding visual salience with ”ease of search.” To mitigate this, we ensure both groups see the pity information on the exact same screen, strictly varying only the visual representation (text vs. bar) rather than the navigational depth required to locate it.

We must guarantee that potential outcomes must be independent of the treatments assigned to *others*, protecting against interferences that could violate the **Stable Unit Treatment Value Assumption (SUTVA)**. This risk primarily stems from social media spillover; if players in the treatment group post screenshots of the ”new bar” on community platforms, players in the control group may become aware of the mechanic, subtly altering their behaviour. Furthermore, we must account for potential cross-over effects if our server-side tagging were to fail, accidentally exposing control users to the treatment. To manage these interference risks, we will rigorously test our server-side assignments prior to launch and actively monitor social channels throughout the test window to assess the extent of any information spillover.

Finally, while we have chosen to conduct this experiment in the field, we acknowledge the methodological downsides of eschewing a laboratory setting. By opting out of the lab, we sacrifice the researcher’s ability to eliminate environmental noise and tightly control for Hawthorne effects (where participants change their behaviour simply because they are in an artificial, observed environment). However, as discussed below, this trade-off is necessary to capture accurate behavioural metrics.

4.2 External validity threats

Although a lab setting prioritises internal control, it fundamentally lacks the authenticity of treatment required for our specific research questions. The evaluation of impulsive financial traits demands an everyday life situation in which real money and high financial stakes are on the line. A lab environment relies heavily on hypothetical intentions that inherently differ from reality and cannot recreate organic, impulsive monetization behaviours. Therefore, a Field Experiment employing **Randomised Controlled Trial** (RCT) logic is essential; it allows us to observe natural, relevant outcomes (such as actual monetization and churn) within the exact environment where these findings will ultimately be deployed.

To further ensure external validity, we must address **selection on returns** driven by the heterogeneity of our player base (Braun and Schwartz, 2025). Different segments of our population will predictably respond differently to visual cues. For instance, ”Whales” (heavy spenders) are highly invested and might already track their pity counts manually, meaning that the new visual bar offers them little to no new utility. Conversely, ”Light Spenders” might find the visual bar highly motivating, significantly altering their spending behaviour. To mitigate this threat and maintain generalisability of our findings, we will stratify our randomisation based on prior spending and engagement levels. This stratification ensures that these high-value units and distinct behavioural groups are perfectly balanced across both the treatment and control groups, preventing our Average Treatment Effect from being artificially skewed by a specific user subset.

Portfolio task 3

Import Data

```
casedata_task3 <- read.csv("Task 3\\CaseData_Task3.csv", stringsAsFactors = FALSE)

# Display the structure of the data to verify column names
str(casedata_task3)
```

```
'data.frame':   5000 obs. of  22 variables:
 $ X1 : num  23.8 21 22.8 25.8 22.9 24.9 24.1 21.4 24.8 23.8 ...
 $ X2 : num  13.7 14.4 11.5 13.5 13.6 12.9 11.8 12.4 14.2 15.9 ...
 $ X3 : num  22.5 21.5 20.6 22.7 21.6 19.4 22.9 19.5 21.6 23.4 ...
 $ X4 : num  16.2 14.5 14.7 15.8 17.5 13.8 17 13.8 15.5 16.8 ...
 $ X5 : int  0 0 0 1 1 0 1 0 0 1 ...
 $ X6 : num  22.9 23.4 20.2 22.5 23.5 21.1 24.6 20 23.9 20 ...
 $ X7 : num  11.6 11.2 9.6 11.5 12.7 9.6 12.9 9.5 11.4 9.2 ...
 $ X8 : num  10.1 12.8 9.7 10.3 12.3 6.3 9.6 7.4 10.6 11.9 ...
 $ X9 : num  18.9 20.2 19 21.9 19.7 15 20.2 17.1 23.8 20.2 ...
 $ X10: num  14.6 13 15.1 18.1 13.7 11.4 15.2 10.3 19.7 14.2 ...
 $ X11: num  13.1 13.7 14.6 14.2 14.5 12 14.1 11.4 16.4 13.6 ...
 $ X12: num  6.2 5.5 6.6 5.3 6.7 3.8 5.9 5 8.8 6.5 ...
 $ X13: num  14 13.5 16.6 13.3 14.8 11.8 13.8 12.9 15.2 15.3 ...
 $ X14: num  5.4 3.7 7.3 4.1 4.2 4.4 4.5 3.7 5.1 5.4 ...
 $ X15: num  11.6 9 13.4 10.2 10.5 10.6 11.8 9.6 11.8 11.7 ...
 $ X16: num  10.9 9.9 12.4 10.6 10.2 9.2 11.2 8.9 11 11.7 ...
 $ X17: num  12 12.4 13.3 11.1 11.8 10.7 12.6 12 11.7 12.9 ...
 $ X18: num  8.6 7.4 8.1 5.9 8.5 6.8 7.5 7.1 6.6 8.1 ...
 $ X19: num  12.3 12.1 11.8 10.3 12.8 10.2 11.1 10.8 11 12 ...
 $ X20: int  0 0 1 1 0 1 0 0 1 0 ...
 $ y  : int  1463 2410 659 2202 2196 2329 1993 2467 1644 1499 ...
```

```
 $ d  : int  0 1 1 1 0 1 1 1 0 1 ...
```

```
# Check the first few rows
head(casedata_task3)
```

```
    X1   X2   X3   X4 X5   X6   X7   X8   X9  X10  X11 X12  X13 X14  X15  X16
1 23.8 13.7 22.5 16.2  0 22.9 11.6 10.1 18.9 14.6 13.1 6.2 14.0 5.4 11.6 10.9
2 21.0 14.4 21.5 14.5  0 23.4 11.2 12.8 20.2 13.0 13.7 5.5 13.5 3.7  9.0  9.9
3 22.8 11.5 20.6 14.7  0 20.2  9.6  9.7 19.0 15.1 14.6 6.6 16.6 7.3 13.4 12.4
4 25.8 13.5 22.7 15.8  1 22.5 11.5 10.3 21.9 18.1 14.2 5.3 13.3 4.1 10.2 10.6
5 22.9 13.6 21.6 17.5  1 23.5 12.7 12.3 19.7 13.7 14.5 6.7 14.8 4.2 10.5 10.2
6 24.9 12.9 19.4 13.8  0 21.1  9.6  6.3 15.0 11.4 12.0 3.8 11.8 4.4 10.6  9.2
   X17 X18  X19 X20    y d
1 12.0 8.6 12.3   0 1463 0
2 12.4 7.4 12.1   0 2410 1
3 13.3 8.1 11.8   1  659 1
4 11.1 5.9 10.3   1 2202 1
5 11.8 8.5 12.8   0 2196 0
6 10.7 6.8 10.2   1 2329 1
```

Estimate the treatment effect using simple linear Regression Adjustment

The estimate for the discount coefficient is the Regression Adjustment ATE

```
model_adj <- lm(y ~ d + X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 + X9 + X10 +
                X11 + X12 + X13 + X14 + X15 + X16 + X17 + X18 + X19 + X20,
                data = casedata_task3)

summary(model_adj)
```

```
Call:
lm(formula = y ~ d + X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 +
    X9 + X10 + X11 + X12 + X13 + X14 + X15 + X16 + X17 + X18 +
    X19 + X20, data = casedata_task3)

Residuals:
    Min      1Q  Median      3Q     Max
-623.40 -102.33    2.64  100.95  521.46

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 4697.0456    74.9246  62.690  < 2e-16 ***
d             15.1536     5.7042   2.657  0.00792 **
```

```
X1             3.0618     1.5140   2.022  0.04320 *
X2            -1.5605     1.8347  -0.851  0.39506
X3             1.3632     1.8200   0.749  0.45391
X4            -2.8476     1.7061  -1.669  0.09516 .
X5            36.2410     5.7218   6.334 2.60e-10 ***
X6            -1.7214     1.7192  -1.001  0.31673
X7            21.8707     1.8286  11.960  < 2e-16 ***
X8            -1.3171     1.8629  -0.707  0.47958
X9             0.8756     1.8314   0.478  0.63260
X10           -1.3391     1.8718  -0.715  0.47440
X11           -0.1072     3.7165  -0.029  0.97700
X12            1.2723     4.1530   0.306  0.75935
X13           -7.1754     3.6384  -1.972  0.04865 *
X14         -304.0026     3.6275 -83.805  < 2e-16 ***
X15         -133.6610     3.7419 -35.720  < 2e-16 ***
X16            0.5139     3.6869   0.139  0.88916
X17            1.5268     3.7039   0.412  0.68019
X18           -1.8884     3.6940  -0.511  0.60923
X19            0.8743     3.3074   0.264  0.79153
X20          -26.9651     5.3907  -5.002 5.87e-07 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 152.5 on 4978 degrees of freedom
Multiple R-squared:  0.8826,    Adjusted R-squared:  0.8821
F-statistic:  1782 on 21 and 4978 DF,  p-value: < 2.2e-16
```

Extract the ATE value from the regression and calculate robust standard errors using the HC3 estimator, which is more reliable in small samples and with heteroskedasticity.

```
coeftest(model_adj, vcov = vcovHC(model_adj, type = "HC3"))
```

```
t test of coefficients:

              Estimate Std. Error  t value  Pr(>|t|)
(Intercept) 4697.04561   76.15859  61.6745 < 2.2e-16 ***
d             15.15365    5.80286   2.6114  0.009044 **
X1             3.06184    1.51318   2.0235  0.043080 *
X2            -1.56049    1.81397  -0.8603  0.389687
X3             1.36316    1.83034   0.7448  0.456453
```

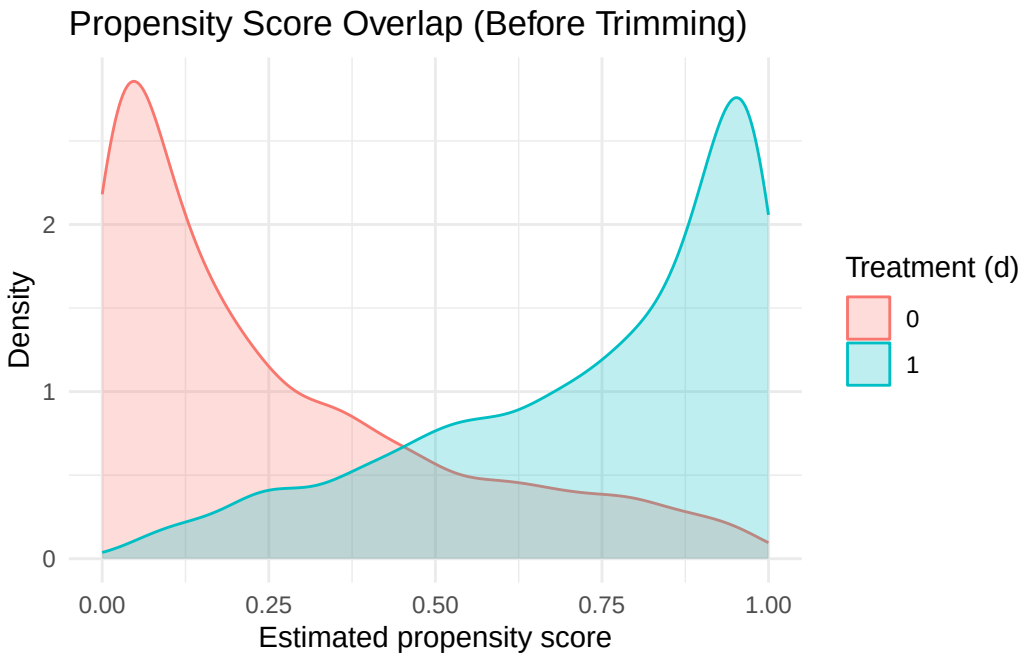
```
X4            -2.84763    1.74765  -1.6294  0.103291
X5            36.24104    5.67533   6.3857 1.861e-10 ***
X6            -1.72139    1.71427  -1.0042  0.315354
X7            21.87065    1.79930  12.1551 < 2.2e-16 ***
X8            -1.31713    1.85035  -0.7118  0.476607
X9             0.87561    1.80426   0.4853  0.627484
X10           -1.33908    1.86892  -0.7165  0.473716
X11           -0.10716    3.77342  -0.0284  0.977345
X12            1.27229    4.16008   0.3058  0.759745
X13           -7.17543    3.72007  -1.9288  0.053807 .
X14         -304.00261    3.64002 -83.5167 < 2.2e-16 ***
X15         -133.66096    3.78412 -35.3216 < 2.2e-16 ***
X16            0.51387    3.54850   0.1448  0.884863
X17            1.52685    3.65959   0.4172  0.676537
X18           -1.88837    3.63423  -0.5196  0.603362
X19            0.87427    3.36898   0.2595  0.795255
X20          -26.96513    5.41454  -4.9801 6.569e-07 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

The estimated Average Treatment Effect (ATE) for the treatment is: 15.15365
```

Propensity Score Estimation and Overlap Diagnostic

Before implementing matching or weighting, I first estimate the propensity scores and evaluate the common support assumption. This step is a necessary prerequisite to ensure that there is sufficient overlap in the covariate distributions between the treated and control groups

```r
# Complete-case sample for IPW variables
ipw_vars <- c("y", "d", paste0("X", 1:20))
df_ipw <- casedata_task3[complete.cases(casedata_task3[, ipw_vars]), ]

# 1) Estimate propensity scores with logistic regression
ps_model <- glm(
  d ~ X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 + X9 + X10 +
      X11 + X12 + X13 + X14 + X15 + X16 + X17 + X18 + X19 + X20,
  data = df_ipw,
  family = binomial(link = "logit")
)

df_ipw$ps <- predict(ps_model, type = "response")

# Overlap/common support diagnostic BEFORE trimming
ggplot(df_ipw, aes(x = ps, color = factor(d), fill = factor(d))) +
```

```
  geom_density(alpha = 0.25) +
  labs(
    title = "Propensity Score Overlap (Before Trimming)",
    x = "Estimated propensity score",
    y = "Density",
    color = "Treatment (d)",
    fill = "Treatment (d)"
  ) +
  theme_minimal()
```



Discussion of Overlap Diagnostic: The density plot of the estimated propensity scores shows that the treated group (d=1) has a distribution of scores that is quite different from the control group (d=0). The treated units tend to have higher propensity scores, while many control units have scores close to zero. This indicates poor overlap between the groups, which can lead to issues with both matching and weighting methods. In particular, IPW may produce extreme weights for control units with very low propensity scores, leading to unstable estimates. Matching may also struggle to find good matches for treated units if the control group is not well represented across the range of propensity scores. This is a red flag that we may need to consider trimming the sample or using more robust methods that are less sensitive to poor overlap, such as AIPW or DML. As such, it is important to consider that the limited overlap is why both matching and IPW weights may struggle with this dataset, and why we should be cautious in interpreting results from those methods.

Matching comparisons

Preparing the data for matching excersises

```
ps_formula <- d ~ X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 + X9 + X10 + X11 +
X12 + X13 + X14 + X15 + X16 + X17 + X18 + X19 + X20
```

1:1 Nearest Neighbor Matching WITHOUT Replacement

```
match_no_replace <- matchit(ps_formula,
                            data = casedata_task3,
                            method = "nearest",
                            distance = "glm",   # Use logistic reg for scores
                            replace = FALSE,    # Without replacement
                            ratio = 1)          # 1:1 matching

# View a summary to check covariate balance (ideally Std. Mean Diff < 0.1)
cat("Balance Summary (Without Replacement):\n")
```

```
Balance Summary (Without Replacement):
```

```
print(summary(match_no_replace, un = FALSE))
```

```
Call:
matchit(formula = ps_formula, data = casedata_task3, method = "nearest",
    distance = "glm", replace = FALSE, ratio = 1)

Summary of Balance for Matched Data:
         Means Treated Means Control Std. Mean Diff. Var. Ratio eCDF Mean
distance        0.7342        0.2657          1.9201     0.9190    0.3878
X1             24.0591       24.0637         -0.0023     0.9305    0.0067
X2             14.1096       14.0910          0.0093     0.9578    0.0048
X3             22.0102       22.0009          0.0047     0.9816    0.0027
X4             15.7842       15.7595          0.0125     0.9536    0.0041
X5              0.5417        0.5324          0.0187          .    0.0093
X6             23.0879       23.0536          0.0173     0.9687    0.0039
X7             11.4389       11.3466          0.0462     1.0201    0.0073
X8             10.4588       10.2308          0.1115     1.0419    0.0180
X9             20.1062       20.4062         -0.1476     0.9862    0.0230
X10            14.7843       15.6086         -0.4177     1.0232    0.0640
X11            13.7279       14.4796         -0.8253     0.9879    0.1073
X12             5.7278        6.8929         -1.5370     1.0032    0.1738
X13            13.9400       14.7616         -0.9285     0.9906    0.1173
X14             4.9429        5.5103         -0.5895     1.0138    0.0846
X15            11.4658       11.8530         -0.3924     1.0262    0.0570
X16            11.3215       11.5974         -0.2730     1.0655    0.0386
```

```
X17            12.3863       12.5703         -0.1844     1.0055    0.0270
X18             7.8680        8.0111         -0.1386     1.0906    0.0202
X19            11.5044       11.5751         -0.0687     1.0678    0.0108
X20             0.3464        0.3493         -0.0060          .    0.0028
         eCDF Max Std. Pair Dist.
distance   0.6293          1.9201
X1         0.0259          1.1445
X2         0.0158          1.1531
X3         0.0105          1.1438
X4         0.0138          1.1503
X5         0.0093          0.9881
X6         0.0170          1.1274
X7         0.0251          1.1152
X8         0.0559          1.1007
X9         0.0713          1.1326
X10        0.1848          1.1405
X11        0.3205          1.2056
X12        0.5543          1.5408
X13        0.3553          1.2431
X14        0.2459          1.1738
X15        0.1572          1.1387
X16        0.1155          1.1115
X17        0.0729          1.1178
X18        0.0583          1.0849
X19        0.0344          1.0910
X20        0.0028          0.9580

Sample Sizes:
          Control Treated
All          2532    2468
Matched      2468    2468
Unmatched      64       0
Discarded       0       0
```

```r
# Extract the matched dataset
data_matched_no_replace <- match.data(match_no_replace)

# Estimate the Treatment Effect
# We include the covariates in the final model to make it "doubly robust"
model_no_replace <- lm(y ~ d + X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 + X9 +
                       X10 + X11 + X12 + X13 + X14 + X15 + X16 + X17 + X18 +
```

```
                        X19 + X20,
                        data = data_matched_no_replace,
                        weights = weights)
```

Looking at the balance table, the results are not great. Many covariates still have SMDs well above 0.1 – X12 is at −1.54, X11 at −0.83, X13 at −0.93. The propensity score distance SMD is 1.92, which is quite high. So even after matching, the treated and control groups still look pretty different on these variables. We think this happens because without replacement, once a good control unit gets matched it's gone, and later treated units get stuck with worse matches. This makes us less confident in the ATT from this specification.

Extract the ATT value and calculate robust standard errors using Variance/Covariance Heteroscedasticity-Consistent matrix estimation, which is used to adjust the standard errors of the estimated treatment effects to account for the matched data structure and potential heteroscedasticity.

```
# Calculate robust standard errors
print(coeftest(model_no_replace, vcov. = vcovHC))
```

```
t test of coefficients:

              Estimate Std. Error  t value  Pr(>|t|)
(Intercept) 4662.91246   76.89579  60.6394 < 2.2e-16 ***
d             17.15026    5.83088   2.9413  0.003284 **
X1             3.06093    1.51975   2.0141  0.044053 *
X2            -1.43813    1.82390  -0.7885  0.430447
X3             1.29676    1.84075   0.7045  0.481171
X4            -2.86753    1.75898  -1.6302  0.103118
X5            36.29551    5.71131   6.3550 2.272e-10 ***
X6            -1.51569    1.71731  -0.8826  0.377500
X7            21.90153    1.80833  12.1115 < 2.2e-16 ***
X8            -1.77083    1.86226  -0.9509  0.341699
X9             0.87361    1.80706   0.4834  0.628801
X10           -1.51174    1.88125  -0.8036  0.421678
X11            0.87193    3.79489   0.2298  0.818284
X12            2.95103    4.24587   0.6950  0.487066
X13           -6.47073    3.73861  -1.7308  0.083553 .
X14         -304.35470    3.65250 -83.3277 < 2.2e-16 ***
X15         -133.08910    3.80349 -34.9913 < 2.2e-16 ***
X16            0.77594    3.58534   0.2164  0.828669
X17            1.03521    3.69715   0.2800  0.779487
```

```
X18           -1.56797    3.65007  -0.4296  0.667525
X19            0.66176    3.39599   0.1949  0.845507
X20          -27.64769    5.43725  -5.0849 3.816e-07 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The estimated Average Treatment Effect on the Treated for the treatment is (ATT): 17.15026

1:1 Nearest Neighbor Matching WITH Replacement

```
match_with_replace <- matchit(ps_formula,
                              data = casedata_task3,
                              method = "nearest",
                              distance = "glm",
                              replace = TRUE,     # With replacement
                              ratio = 1)

# Balance Summary (With Replacement)
print(summary(match_with_replace, un = FALSE))
```

```
Call:
matchit(formula = ps_formula, data = casedata_task3, method = "nearest",
    distance = "glm", replace = TRUE, ratio = 1)

Summary of Balance for Matched Data:
         Means Treated Means Control Std. Mean Diff. Var. Ratio eCDF Mean
distance        0.7342        0.7341          0.0005     0.9942    0.0012
X1             24.0591       24.0335          0.0129     0.8633    0.0137
X2             14.1096       14.0685          0.0206     0.8747    0.0153
X3             22.0102       22.0362         -0.0130     0.8053    0.0165
X4             15.7842       15.7235          0.0308     0.9084    0.0116
X5              0.5417        0.5211          0.0415          .    0.0207
X6             23.0879       22.8316          0.1293     0.9564    0.0208
X7             11.4389       10.9902          0.2247     1.1376    0.0358
X8             10.4588        9.9677          0.2401     1.1778    0.0396
X9             20.1062       19.7199          0.1901     0.9503    0.0292
X10            14.7843       14.5226          0.1326     1.0423    0.0210
X11            13.7279       13.7266          0.0014     1.1714    0.0111
X12             5.7278        5.6479          0.1055     0.8326    0.0147
X13            13.9400       13.8658          0.0838     1.0795    0.0146
X14             4.9429        4.8341          0.1131     0.9741    0.0167
X15            11.4658       11.3919          0.0749     1.0047    0.0125
```

```
X16            11.3215       11.3739         -0.0518     0.9714    0.0117
X17            12.3863       12.3467          0.0398     0.8209    0.0143
X18             7.8680        7.7625          0.1022     0.8899    0.0170
X19            11.5044       11.5057         -0.0013     1.0171    0.0116
X20             0.3464        0.3343          0.0255          .    0.0122
         eCDF Max Std. Pair Dist.
distance   0.0425          0.0027
X1         0.0506          1.1746
X2         0.0523          1.1719
X3         0.0434          1.2037
X4         0.0442          1.1883
X5         0.0207          0.9946
X6         0.0754          1.1502
X7         0.1333          1.1094
X8         0.1264          1.1154
X9         0.1017          1.1269
X10        0.0940          1.0668
X11        0.0425          0.9713
X12        0.0604          0.5051
X13        0.0600          0.9414
X14        0.0802          1.0369
X15        0.0758          1.0793
X16        0.0506          1.1028
X17        0.0409          1.1375
X18        0.0681          1.1460
X19        0.0470          1.1050
X20        0.0122          0.9179

Sample Sizes:
              Control Treated
All           2532.      2468
Matched (ESS)  139.53    2468
Matched        707.      2468
Unmatched     1825.         0
Discarded        0.         0
```

Balance is much better here. Most SMDs drop below 0.13, and the propensity score distance SMD is basically 0 (0.0005). X7 and X8 are still a bit above 0.1 (~0.22–0.24), but overall this is a big improvement over the without-replacement version. This makes sense – allowing replacement means each treated unit can grab the best available control, even if that control has already been used. The downside is that some

controls get reused a lot, which is why we need to cluster the standard errors by subclass below.

Extract the ATT value and calculate robust standard errors clustered by the matched pair ID (`subclass`), which accounts for the fact that some control units may be used multiple times as matches, leading to correlated errors within matched pairs.

```
# Extract the matched dataset
data_matched_with_replace <- get_matches(match_with_replace)

# Estimate the Treatment Effect
model_with_replace <- lm(y ~ d + X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 + X9 +
                         X10 + X11 + X12 + X13 + X14 + X15 + X16 + X17 + X18 +
                         X19 + X20,
                         data = data_matched_with_replace,
                         weights = weights)
print(coeftest(model_with_replace, vcov. = vcovCL, cluster = ~subclass))
```

```
t test of coefficients:

              Estimate Std. Error  t value  Pr(>|t|)
(Intercept) 4397.39206   75.72990  58.0668 < 2.2e-16 ***
d             23.25309    4.38925   5.2977 1.224e-07 ***
X1             3.03662    1.44236   2.1053 0.0353144 *
X2            -2.50465    1.83192  -1.3672 0.1716165
X3             3.88159    1.83070   2.1203 0.0340329 *
X4            -1.66259    1.80731  -0.9199 0.3576577
X5            59.55033    5.81260  10.2450 < 2.2e-16 ***
X6            -5.88580    1.74421  -3.3745 0.0007453 ***
X7            18.50881    1.87071   9.8940 < 2.2e-16 ***
X8             3.07111    1.91503   1.6037 0.1088472
X9             0.44156    1.73320   0.2548 0.7989141
X10           -3.07928    1.86650  -1.6498 0.0990561 .
X11           11.14108    3.66045   3.0436 0.0023497 **
X12          -16.28603    3.85220  -4.2277 2.404e-05 ***
X13            4.06409    3.65274   1.1126 0.2659291
X14         -303.81083    3.75733 -80.8582 < 2.2e-16 ***
X15         -116.73137    4.03738 -28.9126 < 2.2e-16 ***
X16           -5.59644    3.63604  -1.5392 0.1238302
X17            0.87081    3.56892   0.2440 0.8072427
X18            4.16273    3.34672   1.2438 0.2136242
X19           -2.06649    3.37233  -0.6128 0.5400503
```

```
X20          -27.46830    5.55989  -4.9404 8.053e-07 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
The estimated Average Treatment Effect on the Treated for the treatment is (ATT): 23.25309
```

Assessing the Need for Regression Adjustment To evaluate the redundancy of including covariates after matching, I compared the "doubly robust" model (which includes all 20 covariates in the final regression) against a "simple matched model" that uses only the treatment indicator on the matched sample. Discussion of Results: If the matching has achieved high balance (as indicated by my SMD diagnostics showing most variables under 0.13), the estimates from these two models should be nearly identical. A significant difference would suggest that matching was imperfect and the regression adjustment is still working to "clean up" remaining imbalance in the covariates

```r
# Simple matched model without covariates
model_simple_matched <- lm(y ~ d,
                           data = data_matched_with_replace,
                           weights = weights)

# Summary of the simple model
summary(model_simple_matched)
```

```
Call:
lm(formula = y ~ d, data = data_matched_with_replace, weights = weights)

Residuals:
     Min       1Q   Median       3Q      Max
-1497.19  -261.77    12.48   293.48  1139.81

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 1854.516      8.289 223.745   <2e-16 ***
d            -10.329     11.722  -0.881    0.378
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 411.8 on 4934 degrees of freedom
Multiple R-squared:  0.0001574, Adjusted R-squared:  -4.528e-05
F-statistic: 0.7765 on 1 and 4934 DF,  p-value: 0.3782
```

```
Simple Matched ATT: -10.32942
```

```
Doubly Robust ATT: 23.25309
```

As made evident by the highly innacurate ATT measure of the simple matched model, the simple matched model's ATT is quite different from the doubly robust version (-10.3 vs. 23.3), which suggests that the matching alone didn't achieve good balance and the regression adjustment is still doing important work to control for confounding.

Euclidean Distance Matching

```
euc_out <- matchit(
  d ~ X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 + X9 + X10 +
       X11 + X12 + X13 + X14 + X15 + X16 + X17 + X18 + X19 + X20,
  data = casedata_task3,
  method = "nearest",
  distance = "euclidean",
  normalize = TRUE
)

summary(euc_out)
```

```
Call:
matchit(formula = d ~ X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 +
    X9 + X10 + X11 + X12 + X13 + X14 + X15 + X16 + X17 + X18 +
    X19 + X20, data = casedata_task3, method = "nearest", distance = "euclidean",
    normalize = TRUE)

Summary of Balance for All Data:
    Means Treated Means Control Std. Mean Diff. Var. Ratio eCDF Mean eCDF Max
X1        24.0591       24.0646         -0.0028     0.9359    0.0063   0.0248
X2        14.1096       14.0950          0.0073     0.9616    0.0049   0.0155
X3        22.0102       22.0011          0.0046     0.9769    0.0029   0.0106
X4        15.7842       15.7626          0.0109     0.9435    0.0044   0.0134
X5         0.5417        0.5336          0.0164          .    0.0082   0.0082
X6        23.0879       23.0587          0.0147     0.9675    0.0036   0.0147
X7        11.4389       11.3458          0.0467     1.0213    0.0073   0.0240
X8        10.4588       10.2257          0.1140     1.0445    0.0184   0.0577
X9        20.1062       20.4252         -0.1570     0.9849    0.0245   0.0728
X10       14.7843       15.6430         -0.4351     1.0106    0.0667   0.1897
X11       13.7279       14.5120         -0.8609     0.9437    0.1119   0.3290
X12        5.7278        6.9410         -1.6004     0.8800    0.1809   0.5601
X13       13.9400       14.7973         -0.9687     0.9357    0.1224   0.3611
X14        4.9429        5.5363         -0.6165     0.9893    0.0884   0.2546
```

X15	11.4658	11.8701	-0.4097	1.0171	0.0595	0.1626
X16	11.3215	11.6137	-0.2891	1.0480	0.0408	0.1214
X17	12.3863	12.5792	-0.1933	0.9952	0.0283	0.0740
X18	7.8680	8.0182	-0.1455	1.0856	0.0212	0.0596
X19	11.5044	11.5839	-0.0772	1.0663	0.0121	0.0381
X20	0.3464	0.3468	-0.0007	.	0.0003	0.0003

Summary of Balance for Matched Data:

	Means Treated	Means Control	Std. Mean Diff.	Var. Ratio	eCDF Mean	eCDF Max
X1	24.0591	24.0653	-0.0032	0.9349	0.0064	0.0243
X2	14.1096	14.0893	0.0102	0.9621	0.0045	0.0158
X3	22.0102	21.9816	0.0144	0.9787	0.0034	0.0122
X4	15.7842	15.7383	0.0232	0.9592	0.0046	0.0162
X5	0.5417	0.5304	0.0228	.	0.0113	0.0113
X6	23.0879	23.0280	0.0302	0.9705	0.0052	0.0211
X7	11.4389	11.3074	0.0659	1.0282	0.0103	0.0320
X8	10.4588	10.1615	0.1453	1.0810	0.0232	0.0681
X9	20.1062	20.3464	-0.1182	1.0326	0.0185	0.0608
X10	14.7843	15.5527	-0.3893	1.0847	0.0597	0.1771
X11	13.7279	14.4667	-0.8111	1.0295	0.1055	0.3201
X12	5.7278	6.9021	-1.5492	0.9611	0.1752	0.5543
X13	13.9400	14.7636	-0.9307	0.9911	0.1176	0.3553
X14	4.9429	5.5066	-0.5856	1.0280	0.0841	0.2464
X15	11.4658	11.8451	-0.3844	1.0409	0.0558	0.1540
X16	11.3215	11.5895	-0.2652	1.0770	0.0375	0.1126
X17	12.3863	12.5609	-0.1750	1.0136	0.0256	0.0701
X18	7.8680	8.0048	-0.1326	1.0949	0.0193	0.0555
X19	11.5044	11.5732	-0.0668	1.0679	0.0105	0.0340
X20	0.3464	0.3509	-0.0094	.	0.0045	0.0045

	Std. Pair Dist.
X1	0.4711
X2	0.4388
X3	0.4435
X4	0.4554
X5	0.5156
X6	0.4568
X7	0.4445
X8	0.4594
X9	0.4739

```
X10          0.6054
X11          1.0269
X12          1.6159
X13          1.1450
X14          0.8978
X15          0.7994
X16          0.7349
X17          0.7004
X18          0.6556
X19          0.7062
X20          0.6667

Sample Sizes:
          Control Treated
All          2532    2468
Matched      2468    2468
Unmatched      64       0
Discarded       0       0
```

```
# bal.tab(euc_out) - balance info already shown in summary above

m_data_euc <- match.data(euc_out)
lm_euc <- lm(y ~ d + X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 + X9 + X10 + X11 +
             X12 + X13 + X14 + X15 + X16 + X17 + X18 + X19 + X20,
             data = m_data_euc, weights = weights)
```

Extract the ATT value and calculate robust standard errors using the HC3 estimator

```
summary(lm_euc)
```

```
Call:
lm(formula = y ~ d + X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 +
    X9 + X10 + X11 + X12 + X13 + X14 + X15 + X16 + X17 + X18 +
    X19 + X20, data = m_data_euc, weights = weights)

Residuals:
    Min      1Q  Median      3Q     Max
-616.05 -102.44    2.86  101.41  524.23

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
```

```
(Intercept) 4648.7680    75.9523  61.206  < 2e-16 ***
d             16.1581     5.7160   2.827  0.00472 **
X1             3.1540     1.5220   2.072  0.03829 *
X2            -1.8180     1.8464  -0.985  0.32485
X3             1.4143     1.8322   0.772  0.44021
X4            -2.5653     1.7223  -1.489  0.13643
X5            35.9792     5.7585   6.248 4.51e-10 ***
X6            -1.5523     1.7297  -0.897  0.36952
X7            22.1400     1.8412  12.024  < 2e-16 ***
X8            -1.7048     1.8770  -0.908  0.36379
X9             1.1399     1.8412   0.619  0.53587
X10           -1.4086     1.8840  -0.748  0.45470
X11            0.1835     3.7463   0.049  0.96094
X12            1.8764     4.1877   0.448  0.65412
X13           -6.1732     3.6572  -1.688  0.09148 .
X14         -303.7276     3.6391 -83.461  < 2e-16 ***
X15         -133.2866     3.7555 -35.491  < 2e-16 ***
X16            1.1361     3.7119   0.306  0.75956
X17            1.4198     3.7307   0.381  0.70354
X18           -1.8636     3.7137  -0.502  0.61581
X19            1.1463     3.3272   0.345  0.73047
X20          -27.2010     5.4148  -5.023 5.26e-07 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 152.3 on 4914 degrees of freedom
Multiple R-squared:  0.8794,    Adjusted R-squared:  0.8789
F-statistic:  1707 on 21 and 4914 DF,  p-value: < 2.2e-16
```

```r
# Robust Standard Errors (Euclidean Matching):
print(coeftest(lm_euc, vcov. = vcovHC(lm_euc, type = "HC3")))
```

```
t test of coefficients:

              Estimate Std. Error  t value  Pr(>|t|)
(Intercept) 4648.76805   76.81149  60.5218 < 2.2e-16 ***
d             16.15806    5.80282   2.7845  0.005381 **
X1             3.15400    1.52077   2.0740  0.038136 *
X2            -1.81799    1.82728  -0.9949  0.319827
```

```
X3             1.41426    1.84326   0.7673  0.442965
X4            -2.56528    1.76488  -1.4535  0.146144
X5            35.97922    5.71258   6.2982 3.273e-10 ***
X6            -1.55231    1.72393  -0.9004  0.367926
X7            22.13995    1.80807  12.2451 < 2.2e-16 ***
X8            -1.70481    1.86583  -0.9137  0.360918
X9             1.13992    1.81094   0.6295  0.529076
X10           -1.40860    1.88081  -0.7489  0.453934
X11            0.18347    3.80193   0.0483  0.961513
X12            1.87635    4.18418   0.4484  0.653856
X13           -6.17320    3.73096  -1.6546  0.098072 .
X14         -303.72759    3.64576 -83.3099 < 2.2e-16 ***
X15         -133.28657    3.80374 -35.0410 < 2.2e-16 ***
X16            1.13611    3.57993   0.3174  0.750988
X17            1.41977    3.69012   0.3847  0.700440
X18           -1.86365    3.64649  -0.5111  0.609318
X19            1.14632    3.38243   0.3389  0.734697
X20          -27.20100    5.44296  -4.9975 6.010e-07 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Comparing matching quality across methods

Looking at the balance tables side by side, Euclidean distance matching and PSM without replacement perform similarly – and both are pretty bad. The Euclidean SMDs are actually slightly worse on the key problem covariates (X12 = −1.60 vs. −1.54 for PSM, X11 = −0.86 vs. −0.83). Neither method manages to bring X10–X15 anywhere close to the 0.1 threshold. PSM with replacement is clearly the best of the three – it gets most covariates under 0.13 and the propensity score distance is essentially perfectly balanced. So in terms of matching quality, with-replacement PSM wins by a wide margin here, though it comes at the cost of reusing control units and needing clustered standard errors. The poor balance in the other two methods means their ATT estimates should probably be taken with a grain of salt.

On common support: the sample sizes tables show that 64 control units (out of 2532) went unmatched, meaning they had no reasonably close treated counterpart. No treated units were discarded. So we're not losing a huge chunk of the data, which is reassuring – but the fact that those 64 controls couldn't find a match does hint at the limited overlap between the groups, which is also visible in the propensity score density plot in the IPW section. The separation in that plot explains why both the matching balance and the IPW weights struggle here.

Inverse Probability Weighting (IPW)

```r
# 2) Trim propensity scores to reduce extreme weights under weak overlap
df_ipw$ps_trim <- pmin(pmax(df_ipw$ps, 0.01), 0.99)

# Function requested in prompt: two-step IPW ATE with logistic PS + trimming
estimate_ipw_ate <- function(data, trim = 0.01) {
  ps_fit <- glm(
    d ~ X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 + X9 + X10 +
      X11 + X12 + X13 + X14 + X15 + X16 + X17 + X18 + X19 + X20,
    data = data,
    family = binomial(link = "logit")
  )

  ps_hat <- predict(ps_fit, newdata = data, type = "response")
  ps_hat <- pmin(pmax(ps_hat, trim), 1 - trim)

  mean(data$y * (data$d / ps_hat - (1 - data$d) / (1 - ps_hat)))
}

# 3) Manual IPW ATE using slide formula:
#    ATE = mean( y * ( d/ps - (1-d)/(1-ps) ) )
ate_ipw_manual <- estimate_ipw_ate(df_ipw, trim = 0.01)

cat("\nManual IPW ATE (primary estimate):", round(ate_ipw_manual, 4), "\n")
```

```
Manual IPW ATE (primary estimate): -318.1235
```

```r
# 4) Bootstrap inference for the full two-step IPW estimator:
#    re-sample rows, re-estimate propensity model, re-calculate ATE each draw.
set.seed(123)
B_ipw <- 1000
n_ipw <- nrow(df_ipw)
boot_ipw_ate <- numeric(B_ipw)

for (b in seq_len(B_ipw)) {
  idx <- sample.int(n_ipw, size = n_ipw, replace = TRUE)
  boot_data <- df_ipw[idx, ]
  boot_ipw_ate[b] <- estimate_ipw_ate(boot_data, trim = 0.01)
}

ipw_boot_mean <- mean(boot_ipw_ate)
ipw_boot_se <- sd(boot_ipw_ate)
ipw_boot_ci <- quantile(boot_ipw_ate, probs = c(0.025, 0.975))
```

Table 1
IPW Bootstrap Results

Statistic	Value
Bootstrap Mean (IPW)	-315.5704
Bootstrap SE (IPW)	81.2465
95% Percentile CI (IPW)	[-487.5767, -173.1673]

```
# 5) Secondary comparison: IPW weighted regression-adjustment model
df_ipw$ipw_weight <- ifelse(df_ipw$d == 1,
                            1 / df_ipw$ps_trim,
                            1 / (1 - df_ipw$ps_trim))

# Inverse Probability Weights Summary:
summary(df_ipw$ipw_weight)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  1.010   1.059   1.216   1.972   1.708 100.000
```

```
#Secondary comparison: weighted lm IPW-RA model
lm_ipw <- lm(y ~ d + X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 + X9 + X10 + X11 +
             X12 + X13 + X14 + X15 + X16 + X17 + X18 + X19 + X20,
             data = df_ipw, weights = ipw_weight)

print(summary(lm_ipw))
```

```
Call:
lm(formula = y ~ d + X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 +
    X9 + X10 + X11 + X12 + X13 + X14 + X15 + X16 + X17 + X18 +
    X19 + X20, data = df_ipw, weights = ipw_weight)

Weighted Residuals:
     Min       1Q   Median       3Q      Max
-2078.34  -118.35    10.78   128.11  1164.02

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 4590.0109    74.6317  61.502  < 2e-16 ***
d             15.8064     4.3103   3.667 0.000248 ***
X1            -0.2598     1.4704  -0.177 0.859763
X2             0.9350     1.8075   0.517 0.604976
```

```
X3             2.3339     1.7892   1.304 0.192136
X4            -3.0046     1.7406  -1.726 0.084373 .
X5            58.9206     5.7048  10.328  < 2e-16 ***
X6            -3.5986     1.7019  -2.114 0.034523 *
X7            20.1744     1.8480  10.917  < 2e-16 ***
X8            -0.7299     1.8412  -0.396 0.691829
X9             2.6665     1.8280   1.459 0.144698
X10           -1.1871     1.8626  -0.637 0.523939
X11            2.3650     3.6798   0.643 0.520437
X12           -3.9171     3.7564  -1.043 0.297095
X13           -4.4726     3.6656  -1.220 0.222462
X14         -308.5079     3.7113 -83.127  < 2e-16 ***
X15         -122.9808     3.8228 -32.170  < 2e-16 ***
X16            2.2777     3.7034   0.615 0.538567
X17           -4.1136     3.5600  -1.156 0.247939
X18           -2.1922     3.5469  -0.618 0.536563
X19            4.5555     3.2707   1.393 0.163743
X20          -23.7826     5.4203  -4.388 1.17e-05 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 211.6 on 4978 degrees of freedom
Multiple R-squared:  0.889, Adjusted R-squared:  0.8885
F-statistic:  1898 on 21 and 4978 DF,  p-value: < 2.2e-16
```

```
print(coeftest(lm_ipw, vcov. = vcovHC(lm_ipw, type = "HC3")))
```

```
t test of coefficients:

              Estimate Std. Error  t value  Pr(>|t|)
(Intercept) 4590.01087  131.68226  34.8567 < 2.2e-16 ***
d             15.80639    8.30433   1.9034  0.057047 .
X1            -0.25979    2.54310  -0.1022  0.918637
X2             0.93500    3.28803   0.2844  0.776141
X3             2.33393    3.63634   0.6418  0.521010
X4            -3.00456    3.21316  -0.9351  0.349793
X5            58.92058    9.94823   5.9227 3.379e-09 ***
X6            -3.59856    3.63202  -0.9908  0.321837
X7            20.17440    2.96300   6.8088 1.101e-11 ***
```

```
X8            -0.72987    3.38596  -0.2156  0.829343
X9             2.66654    3.20263   0.8326  0.405105
X10           -1.18711    3.30364  -0.3593  0.719361
X11            2.36505    5.89540   0.4012  0.688313
X12           -3.91711    5.88667  -0.6654  0.505812
X13           -4.47265    5.68332  -0.7870  0.431332
X14         -308.50795    6.23565 -49.4748 < 2.2e-16 ***
X15         -122.98077    6.88727 -17.8562 < 2.2e-16 ***
X16            2.27770    5.59034   0.4074  0.683705
X17           -4.11359    7.77266  -0.5292  0.596663
X18           -2.19218    6.65799  -0.3293  0.741977
X19            4.55546    5.93037   0.7682  0.442430
X20          -23.78255    8.47568  -2.8060  0.005036 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

IPW Critical Discussion Points

The manual IPW estimate came out at around −318, which obviously doesn't make sense for a sales effect and is likely due to its sensitivity to poor overlap. The weighted regression version (~15.81) does much better because the regression component doesn't rely purely on the weights. It also shows why AIPW and DML are generally more appealing: they don't put all their eggs in the weighting basket.

Augmented Inverse Probability Weighting (AIPW) - Manual Estimation

This section implements AIPW manually using the doubly robust score:

$$ATE = \frac{1}{N}\sum_{i=1}^{N}\left(\frac{T_i}{\hat{p}(X_i)}(Y_i - \hat{\mu}_{T=1}(X_i)) + \hat{\mu}_{T=1}(X_i)\right) - \frac{1}{N}\sum_{i=1}^{N}\left(\frac{(1-T_i)}{1-\hat{p}(X_i)}(Y_i - \hat{\mu}_{T=0}(X_i)) + \hat{\mu}_{T=0}(X_i)\right)$$

```r
df <- casedata_task3
df$treatment <- df$d
df$sales <- df$y
covariates <- paste0("X", 1:20)

# Basic checks
stopifnot(all(c("treatment", "sales") %in% names(df)))
stopifnot(all(covariates %in% names(df)))
stopifnot(all(na.omit(unique(df$treatment)) %in% c(0, 1)))

# Keep complete cases for variables used in AIPW
aipw_vars <- c("treatment", "sales", covariates)
df_aipw <- df[complete.cases(df[, aipw_vars]), aipw_vars]
```

```r
# AIPW function returning the sample ATE using the manual formula
estimate_aipw_ate <- function(data, covariate_names, trim = 0.01) {
  # 1) Selection model (propensity score): P(T=1|X)
  ps_formula <- as.formula(
    paste("treatment ~", paste(covariate_names, collapse = " + "))
  )
  ps_model <- glm(ps_formula, data = data, family = binomial(link = "logit"))
  p_hat <- predict(ps_model, newdata = data, type = "response")

  # Trimming avoids extreme inverse-probability weights when overlap is weak
  p_hat <- pmin(pmax(p_hat, trim), 1 - trim)

  # 2) Outcome models fitted separately by treatment arm
  mu1_formula <- as.formula(
    paste("sales ~", paste(covariate_names, collapse = " + "))
  )
  mu0_formula <- mu1_formula

  outcome_model_treated <- lm(mu1_formula, data = data[data$treatment == 1, ])
  outcome_model_control <- lm(mu0_formula, data = data[data$treatment == 0, ])

  # Predicted potential outcomes for every unit
  mu1_hat <- predict(outcome_model_treated, newdata = data)
  mu0_hat <- predict(outcome_model_control, newdata = data)

  # 3) Doubly robust correction terms:
  #    residuals (Y - mu_hat) are reweighted by inverse propensity scores.
  #    If either the PS model or the outcome models are correctly specified,
  #    the estimator remains consistent.
  score_treated <- (data$treatment / p_hat) * (data$sales - mu1_hat) + mu1_hat
  score_control <- ((1 - data$treatment) /
                    (1 - p_hat)) * (data$sales - mu0_hat) + mu0_hat

  mean(score_treated) - mean(score_control)
}

# Point estimate
aipw_ate <- estimate_aipw_ate(df_aipw, covariates, trim = 0.01)
cat("Manual AIPW ATE estimate:", round(aipw_ate, 4), "\n")
```

```
Manual AIPW ATE estimate: 18.8487
```

```r
# 4) Bootstrap inference for AIPW
set.seed(123)
```

```r
B <- 1000
n <- nrow(df_aipw)
boot_ate <- numeric(B)

for (b in seq_len(B)) {
  idx <- sample.int(n, size = n, replace = TRUE)
  boot_data <- df_aipw[idx, ]
  boot_ate[b] <- estimate_aipw_ate(boot_data, covariates, trim = 0.01)
}

aipw_se <- sd(boot_ate)
aipw_ci <- quantile(boot_ate, probs = c(0.025, 0.975))
aipw_boot_mean <- mean(boot_ate)
```

Table 2
AIPW Bootstrap Results

Statistic	Value
Bootstrap Mean (AIPW)	19.2227
Bootstrap SE (AIPW)	8.4778
95% Percentile CI (AIPW)	[2.1545, 35.7856]

AIPW Critical Discussion Points

- Doubly robust consistency: AIPW is consistent if either the propensity score model or the outcome regression models are correctly specified.
- Efficiency: Compared with simple IPW, AIPW is typically more stable and can achieve the semiparametric efficiency bound under correct specification.
- Debiasing logic: The residual-weighted terms, $(Y - \hat{\mu})$, correct the regression predictions using inverse-probability weighting.
- Overlap/common support: Propensity scores should stay away from 0 and 1; weak overlap creates extreme weights and unstable estimates, so diagnostics and trimming are important.

Double Machine Learning (DML): PLR with Cross-Fitting

We now estimate the ATE with DoubleML using the partially linear regression model:

$$Y = \beta D + g(X) + \epsilon, \qquad D = m(X) + \xi$$

Cross-fitting (sample splitting) is used to reduce overfitting bias in nuisance estimation, and the orthogonal score makes the final β less sensitive to small nuisance-model errors.

```r
# Data setup for DML
x_cols <- paste0("X", 1:20)
dml_vars <- c("y", "d", x_cols)
dml_df <- casedata_task3[complete.cases(casedata_task3[, dml_vars]), dml_vars]

# Binary treatment check
stopifnot(all(unique(dml_df$d) %in% c(0, 1)))

# DoubleML data backend
dml_data <- DoubleMLData$new(
  data = dml_df,
  y_col = "y",
  d_cols = "d",
  x_cols = x_cols
)

# Tasks for out-of-sample nuisance performance
# Use only X covariates as predictors to match DML nuisance definitions.
task_y_data <- dml_df[, c("y", x_cols)]
task_y <- TaskRegr$new(id = "outcome_task", backend = task_y_data, target = "y")

task_d_data <- dml_df[, x_cols]
task_d_data$d_factor <- factor(dml_df$d, levels = c(0, 1))
task_d <- TaskClassif$new(
  id = "treatment_task",
  backend = task_d_data,
  target = "d_factor",
  positive = "1"
)

# Helper to construct requested learners with a fallback if IDs differ by install
get_lrn <- function(primary_id, fallback_id = NULL, ...) {
  if (primary_id %in% mlr_learners$keys()) {
    return(lrn(primary_id, ...))
  }
  if (!is.null(fallback_id) && fallback_id %in% mlr_learners$keys()) {
    message("Learner ", primary_id, " not found; using fallback ", fallback_id)
    return(lrn(fallback_id, ...))
  }
  stop("Required learner not found: ", primary_id,
       if (!is.null(fallback_id))
       paste0(" (fallback also missing: ", fallback_id, ")")
       else "")
}
```

```
# Run one DML specification: nuisance metrics + PLR estimate
run_dml_spec <- function(spec_name, regr_id, classif_id,
                         regr_fallback = NULL, classif_fallback = NULL) {
  set.seed(123)

  # Outcome nuisance model g(X): evaluate out-of-sample RMSE
  learner_y <- get_lrn(regr_id, regr_fallback)
  rr_y <- resample(task_y,
          learner_y$clone(),
          rsmp("cv", folds = 5),
          store_models = FALSE)
  rmse_y <- rr_y$aggregate(msr("regr.rmse"))

  # Selection nuisance model m(X): evaluate out-of-sample log-loss and error
  learner_d_class <- get_lrn(classif_id,
                             classif_fallback,
                             predict_type = "prob")
  rr_d <- resample(task_d,
                   learner_d_class$clone(),
                   rsmp("cv", folds = 5),
                   store_models = FALSE)
  logloss_d <- rr_d$aggregate(msr("classif.logloss"))
  ce_d <- rr_d$aggregate(msr("classif.ce"))

  # PLR in DoubleML uses regression nuisance learners for both l(X) and m(X)
  learner_m_regr <- get_lrn(regr_id, regr_fallback)
  dml_plr <- DoubleMLPLR$new(
    data = dml_data,
    ml_l = learner_y$clone(),
    ml_m = learner_m_regr$clone(),
    n_folds = 5
  )
  dml_plr$fit()

  data.frame(
    Model = spec_name,
    ATE_Beta = as.numeric(dml_plr$coef),
    Std_Error = as.numeric(dml_plr$se),
    RMSE_Y = as.numeric(rmse_y),
    LogLoss_D = as.numeric(logloss_d),
    Classif_Error_D = as.numeric(ce_d),
    stringsAsFactors = FALSE
  )
```

```
}

# Three requested learner families
dml_lasso <- run_dml_spec(
  spec_name = "Lasso",
  regr_id = "regr.glmnet",
  classif_id = "classif.lasso",
  regr_fallback = "regr.cv_glmnet",
  classif_fallback = "classif.cv_glmnet"
)

dml_rf <- run_dml_spec(
  spec_name = "Random Forest",
  regr_id = "regr.ranger",
  classif_id = "classif.ranger"
)

dml_xgb <- run_dml_spec(
  spec_name = "XGBoost",
  regr_id = "regr.xgboost",
  classif_id = "classif.xgboost"
)

dml_results <- rbind(dml_lasso, dml_rf, dml_xgb)
dml_results$CI_Lower_95 <- dml_results$ATE_Beta - 1.96 * dml_results$Std_Error
dml_results$CI_Upper_95 <- dml_results$ATE_Beta + 1.96 * dml_results$Std_Error

cat("\n=== DML (PLR) Comparison Across Learners ===\n\n")
```

```
=== DML (PLR) Comparison Across Learners ===
```

```
dml_results_to_print <- dml_results
num_cols <- sapply(dml_results_to_print, is.numeric)
dml_results_to_print[num_cols] <- lapply(dml_results_to_print[num_cols], round, 4)
print(dml_results_to_print, row.names = FALSE)
```

```
         Model ATE_Beta Std_Error   RMSE_Y LogLoss_D Classif_Error_D
         Lasso  15.2253    5.7710 152.7603    0.4170          0.1928
 Random Forest  13.6981    6.1429 164.2681    0.4429          0.2040
       XGBoost  18.3478    5.8701 170.4984    0.8409          0.2206
 CI_Lower_95 CI_Upper_95
      3.9140     26.5365
      1.6579     25.7383
```

```
      6.8425      29.8531
```

```r
# "Well-fitting" model selection rule:
# prioritize low out-of-sample nuisance errors (RMSE_Y and LogLoss_D)
scale01 <- function(x) {
  rng <- range(x)
  if (diff(rng) == 0) return(rep(0, length(x)))
  (x - rng[1]) / diff(rng)
}

dml_results$fit_score <- scale01(dml_results$RMSE_Y) +
                         scale01(dml_results$LogLoss_D)
best_idx <- which.min(dml_results$fit_score)

cat("\nBest-fitting nuisance combination (lower RMSE + LogLoss):",
    dml_results$Model[best_idx], "\n")
```

```
Best-fitting nuisance combination (lower RMSE + LogLoss): Lasso
```

DML Discussion Points

- DML solves regularization bias and overfitting bias: sample splitting and cross-fitting reduce overfitting from flexible learners.
- Predictive performance of nuisance models is central: lower RMSE/log-loss means cleaner residualization (orthogonalization) for treatment-effect estimation.
- Compared with weighting estimators, PLR-style partialling out avoids direct division by propensity scores, which can be more stable under weak overlap.
- Even with ML-based nuisance models, causal validity still depends on conditional independence and excluding bad controls (e.g., colliders).

Comparison of Treatment Effects Across Methods

```r
# Extract treatment effects and standard errors from all methods
ate_regression <- coef(model_adj)["d"]
se_regression <- sqrt(diag(vcovHC(model_adj, type = "HC3")))["d"]

att_no_replace <- coef(model_no_replace)["d"]
se_no_replace <- sqrt(diag(vcovHC(model_no_replace, type = "HC3")))["d"]

att_with_replace <- coef(model_with_replace)["d"]
se_with_replace <- sqrt(diag(vcovCL(model_with_replace,cluster=~subclass)))["d"]

ate_euc <- coef(lm_euc)["d"]
se_euc <- sqrt(diag(vcovHC(lm_euc, type = "HC3")))["d"]
```

```r
ate_ipw <- ate_ipw_manual
se_ipw <- ipw_boot_se

ate_ipw_lm <- coef(lm_ipw)["d"]
se_ipw_lm <- sqrt(diag(vcovHC(lm_ipw, type = "HC3")))["d"]

ate_aipw <- aipw_ate
se_aipw <- aipw_se

# Calculate 95% confidence intervals
ci_lower_reg <- ate_regression - 1.96 * se_regression
ci_upper_reg <- ate_regression + 1.96 * se_regression

ci_lower_no_rep <- att_no_replace - 1.96 * se_no_replace
ci_upper_no_rep <- att_no_replace + 1.96 * se_no_replace

ci_lower_rep <- att_with_replace - 1.96 * se_with_replace
ci_upper_rep <- att_with_replace + 1.96 * se_with_replace

ci_lower_euc <- ate_euc - 1.96 * se_euc
ci_upper_euc <- ate_euc + 1.96 * se_euc

ci_lower_ipw <- ipw_boot_ci[1]
ci_upper_ipw <- ipw_boot_ci[2]

ci_lower_ipw_lm <- ate_ipw_lm - 1.96 * se_ipw_lm
ci_upper_ipw_lm <- ate_ipw_lm + 1.96 * se_ipw_lm

ci_lower_aipw <- aipw_ci[1]
ci_upper_aipw <- aipw_ci[2]

# Create base comparison table for classical/weighting estimators
comparison_table_base <- data.frame(
  Method = c(
    "Regression Adjustment",
    "Matching (1:1 No Replacement)",
    "Matching (1:1 With Replacement)",
    "Euclidean Distance Matching",
    "Inverse Probability Weighting (IPW) - Manual Formula",
    "IPW Weighted Regression (Secondary)",
    "Augmented IPW (AIPW) - Manual Formula"
  ),
  Estimand = c(
    "ATE",
```

```
    "ATT",
    "ATT",
    "ATT",
    "ATE",
    "ATE",
    "ATE"
  ),
  Inference = c(
    "HC3",
    "HC3",
    "Clustered (subclass)",
    "HC3",
    "Bootstrap (1000)",
    "HC3",
    "Bootstrap (1000)"
  ),
  Estimate = c(
    round(ate_regression, 4),
    round(att_no_replace, 4),
    round(att_with_replace, 4),
    round(ate_euc, 4),
    round(ate_ipw, 4),
    round(ate_ipw_lm, 4),
    round(ate_aipw, 4)
  ),
  Std_Error = c(
    round(se_regression, 4),
    round(se_no_replace, 4),
    round(se_with_replace, 4),
    round(se_euc, 4),
    round(se_ipw, 4),
    round(se_ipw_lm, 4),
    round(se_aipw, 4)
  ),
  CI_Lower = c(
    round(ci_lower_reg, 4),
    round(ci_lower_no_rep, 4),
    round(ci_lower_rep, 4),
    round(ci_lower_euc, 4),
    round(ci_lower_ipw, 4),
    round(ci_lower_ipw_lm, 4),
    round(ci_lower_aipw, 4)
  ),
  CI_Upper = c(
```

```
    round(ci_upper_reg, 4),
    round(ci_upper_no_rep, 4),
    round(ci_upper_rep, 4),
    round(ci_upper_euc, 4),
    round(ci_upper_ipw, 4),
    round(ci_upper_ipw_lm, 4),
    round(ci_upper_aipw, 4)
  ),
  row.names = NULL
)

# Add DML learner-specific estimates to the same comparison table
dml_comparison <- data.frame(
  Method = paste0("DML-PLR (", dml_results$Model, ")"),
  Estimand = rep("ATE", nrow(dml_results)),
  Inference = rep("Orthogonal score SE (cross-fit)", nrow(dml_results)),
  Estimate = round(dml_results$ATE_Beta, 4),
  Std_Error = round(dml_results$Std_Error, 4),
  CI_Lower = round(dml_results$CI_Lower_95, 4),
  CI_Upper = round(dml_results$CI_Upper_95, 4),
  row.names = NULL
)

comparison_table <- rbind(comparison_table_base, dml_comparison)
```

```
=== Summary: Treatment Effects Across All Methods (with 95% CIs) ===
```

Method	Estimand	Inference	Estimate	Std_Error	CI_Lower	CI_Upper
Regression Adjustment	ATE	HC3	15.1536	5.8029	3.7800	26.5273
Matching (1:1 No Replacement)	ATT	HC3	17.1503	5.8309	5.7217	28.5788
Matching (1:1 With Replacement)	ATT	Clustered (subclass)	23.2531	4.3893	14.6502	31.8560
Euclidean Distance Matching	ATT					
Inverse Probability Weighting (IPW) - Manual Formula	ATE					
IPW Weighted Regression (Secondary)	ATE					
Augmented IPW (AIPW) - Manual Formula	ATE					
DML-PLR (Lasso)	ATE					
DML-PLR (Random Forest)	ATE					
DML-PLR (XGBoost)	ATE					

HC3	16.1581	5.8028	4.7845	27.5316
Bootstrap (1000)	-318.1235	81.2465	-487.5767	-173.1673
HC3	15.8064	8.3043	-0.4701	32.0829
Bootstrap (1000)	18.8487	8.4778	2.1545	35.7856
Orthogonal score SE (cross-fit)	15.2253	5.7710	3.9140	26.5365
Orthogonal score SE (cross-fit)	13.6981	6.1429	1.6579	25.7383
Orthogonal score SE (cross-fit)	18.3478	5.8701	6.8425	29.8531

[1]

Discussion

One thing worth noting upfront is that every method used here relies on the same assumption: that the 20 covariates are enough to account for all the confounding between who gets the discount and how much they buy. We can't really test this – if there's something unobserved driving both treatment and sales (e.g., customer motivation), all of these estimates could be off. But given that we have 20 covariates covering demographics and purchasing behavior, it seems like a reasonable assumption for this dataset.

1. Comparison of ATE vs. ATT Estimates

An important thing to keep in mind when comparing the results is that not all methods estimate the same thing.

- **ATE methods** (Regression, IPW, AIPW, DML) estimate the effect if the discount were given to everyone. Ignoring the broken manual IPW, these range from roughly **13.70 to 18.85**.
- **ATT methods** (Matching) estimate the effect for the customers who actually got the discount, ranging from about **16.16 to 23.25**.

The ATT estimates being higher than the ATE could suggest that the customers who received the discount also happen to respond more strongly to it – something like selection on returns. Though we're not 100% sure this isn't partly driven by differences in how the methods handle the data (e.g., the poor balance in the without-replacement matching might inflate things).

---

2. Methodological Divergence and Instability

- **IPW blowup:** The manual IPW gave −318, which is nonsense. we already discussed this above – it's an overlap problem with extreme weights. The weighted regression IPW (~15.81) is much more reasonable since it doesn't rely only on the weights.
- **Matching differences:** The without-replacement estimate (17.15) and with-replacement estimate (23.25) are pretty far apart. Given the balance diagnostics, we'd trust the with-replacement version more – the without-replacement matching left large SMDs on X10–X14, so those matched pairs weren't really comparable. The with-replacement version got balance mostly under control, though

[1] Note: ATT rows are matching estimands; all other rows report ATE. DML rows include learner-specific PLR estimates with 5-fold cross-fitting and orthogonal-score inference.

it comes with more variance from reusing controls.

---

3. The Role of Double Robustness and Machine Learning

- **AIPW (18.85)** is doubly robust – it stays consistent if either the propensity score model or the outcome model is right. In theory it should also be more efficient than plain IPW.
- **DML** results vary by learner:
  - XGBoost: 18.35
  - Lasso: 15.23
  - Random Forest: 13.70

The fact that XGBoost and AIPW land in a similar range (~18) is interesting – it might mean there are some non-linearities in the covariate-outcome relationship that OLS misses but these methods pick up. The Random Forest estimate being lower is a bit puzzling and could reflect differences in how well the nuisance models fit.

---

4. Conclusion: Which Estimate to Trust?

We'd lean toward trusting the **AIPW** or **DML-XGBoost** estimates, for a few reasons:

1. They don't assume linearity the way plain regression does.
2. AIPW has the double robustness property, which gives some insurance against model misspecification.
3. They use the full dataset rather than discarding unmatched units, so they're answering the ATE question more directly.

That said, none of these methods can fully overcome the unconfoundedness assumption. If there are important unobserved confounders, all estimates could be biased.

Suggestions for the Marketing Manager

Across all the credible methods, the discount seems to increase sales by roughly $15–19 per customer (ATE). The campaign appears to work. Whether it's worth expanding depends on the cost of the discount itself – if the discount costs less than $15 per customer, broader rollout looks profitable.

For the customers currently being targeted, the effect might be even larger (~$17–23 based on matching ATT). So the existing targeting seems to be picking customers who respond well.

If the company wants to be more confident before scaling up, running a proper randomized experiment on a sample of untargeted customers would help. That would give a clean ATE estimate without having to worry about whether the covariates capture everything. For now, the observational evidence is encouraging but comes with the usual caveat that we're assuming the data accounts for all relevant confounders.

Reflection on AI Use

We used AI tools at various points during this assignment, mainly in a supporting role rather than for generating the analysis or writing from scratch.

Tools used: Primarily Google Gemini for code-related tasks, and NotebookLM for revisiting theoretical concepts from the course material.

What we used AI for:

- Debugging code when we ran into errors or unexpected output, and getting suggestions on how to improve code quality (e.g., fixing syntax issues, making sure we were using the right functions from MatchIt or DoubleML).
- Checking for oversights in our analysis – essentially using it as a second pair of eyes to see if we missed something obvious or made a methodological mistake.
- Searching for information to refresh our understanding of the theory behind the methods, particularly around IPW, AIPW, and how DML cross-fitting works.
- NotebookLM was useful for going back over course readings and lecture material to make sure we had the concepts right before writing up the discussion.

How we handled AI input: We treated AI suggestions as starting points rather than final answers. When it flagged something or suggested a change, I checked it against the course material and our own understanding before incorporating it. The analytical choices, interpretations, and written discussion are our own.

Portfolio task 4

Import Data

```
df <- read_delim("Store_data_2026_taks4.csv", delim = ";")

# Create necessary dummy variables
df <- df %>%
  mutate(
    # 'post_treatment' is 1 for the weeks after the marketing strategy started
    post_treatment = ifelse(Weekind >= 78, 1, 0),
    # 'treated' is the actual interaction / DiD term (1 ONLY for Store 109 AND Weekind >= 78)
    treated = ifelse(storeNum == 109 & Weekind >= 78, 1, 0),
    # Variable identifying the treated store (regardless of time)
    is_treated_store = ifelse(storeNum == 109, 1, 0)
  )

# Display the structure of the data to verify column names
str(df)
```

```
tibble [2,184 x 18] (S3: tbl_df/tbl/data.frame)
 $ storeNum        : num [1:2184] 101 101 101 101 101 101 101 101 101 101 ...
```

```
 $ Year            : num [1:2184] 2024 2024 2024 2024 2024 ...
 $ Week            : num [1:2184] 1 2 3 4 5 6 7 8 9 10 ...
 $ Date            : Date[1:2184], format: "2024-01-08" "2024-01-15" ...
 $ Weekind         : num [1:2184] 1 2 3 4 5 6 7 8 9 10 ...
 $ p1sales         : num [1:2184] 457 500 456 450 454 449 453 450 498 458 ...
 $ p2sales         : num [1:2184] 588 707 591 591 590 593 710 591 594 592 ...
 $ p1price         : num [1:2184] 2.49 2.99 2.19 2.29 2.19 2.79 2.49 2.99 2.79 2.79 ...
 $ p2price         : num [1:2184] 2.99 3.19 2.99 3.19 3.19 2.59 3.19 3.19 2.99 3.19 ...
 $ p1prom          : num [1:2184] 0 1 0 0 0 0 0 0 1 0 ...
 $ p2prom          : num [1:2184] 0 1 0 0 0 0 1 0 0 0 ...
 $ compind         : num [1:2184] 0.123 0.123 0.123 0.123 0.123 0.123 0.123 0.123 0.123 0.123
 $ storesize       : num [1:2184] 143 143 143 143 143 143 143 143 143 143 ...
 $ city            : chr [1:2184] "OSLO" "OSLO" "OSLO" "OSLO" ...
 $ citysize        : num [1:2184] 730000 730000 730000 730000 730000 730000 730000 730000 7300
 $ post_treatment  : num [1:2184] 0 0 0 0 0 0 0 0 0 0 ...
 $ treated         : num [1:2184] 0 0 0 0 0 0 0 0 0 0 ...
 $ is_treated_store: num [1:2184] 0 0 0 0 0 0 0 0 0 0 ...
```

```
# Check the first few rows
head(df)
```

```
# A tibble: 6 x 18
  storeNum  Year  Week Date       Weekind p1sales p2sales p1price p2price p1prom
     <dbl> <dbl> <dbl> <date>       <dbl>   <dbl>   <dbl>   <dbl>   <dbl>  <dbl>
1      101  2024     1 2024-01-08       1     457     588    2.49    2.99      0
2      101  2024     2 2024-01-15       2     500     707    2.99    3.19      1
3      101  2024     3 2024-01-22       3     456     591    2.19    2.99      0
4      101  2024     4 2024-01-29       4     450     591    2.29    3.19      0
5      101  2024     5 2024-02-05       5     454     590    2.19    3.19      0
6      101  2024     6 2024-02-12       6     449     593    2.79    2.59      0
# i 8 more variables: p2prom <dbl>, compind <dbl>, storesize <dbl>, city <chr>,
#   citysize <dbl>, post_treatment <dbl>, treated <dbl>, is_treated_store <dbl>
```

A simple pre-post analysis of only the treated store for each product controlling for relevant covariates

The relevant covariates chosen are price and promotion for each product, as they are likely to influence sales. Time-invariant covariates such as store size and city size are excluded from this analysis sicne these stable factors are automatically controlled without requiring additional covariates. For this method to output an appropriate causal estimate, we would need to assume that no other external factors influenced sales at Store 109 between the pre- and post-treatment weeks. (This is a strong and unlikely assumption for this dataset, which is why we expect that more advanced methods like Two-Way Fixed Effects or Synthetic Control are generally more appropriate for establishing causality)

```r
# Filter data to only include Store 109
df_109 <- df %>% filter(storeNum == 109)

# --- Product 1: Pre-Post Model ---
pre_post_p1 <- lm(p1sales ~ post_treatment + p1price + p1prom, data = df_109)
summary(pre_post_p1)
```

```
Call:
lm(formula = p1sales ~ post_treatment + p1price + p1prom, data = df_109)

Residuals:
    Min      1Q  Median      3Q     Max
-23.202  -4.770   1.284   4.793  16.628

Coefficients:
               Estimate Std. Error t value Pr(>|t|)
(Intercept)    320.5430     6.3717  50.307   <2e-16 ***
post_treatment  33.6894     1.7107  19.693   <2e-16 ***
p1price         -0.4704     2.4406  -0.193    0.848
p1prom          35.2861     2.0358  17.332   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 7.615 on 100 degrees of freedom
Multiple R-squared:  0.8839,    Adjusted R-squared:  0.8804
F-statistic: 253.7 on 3 and 100 DF,  p-value: < 2.2e-16
```

```r
# --- Product 2: Pre-Post Model ---
pre_post_p2 <- lm(p2sales ~ post_treatment + p2price + p2prom, data = df_109)
summary(pre_post_p2)
```

```
Call:
lm(formula = p2sales ~ post_treatment + p2price + p2prom, data = df_109)

Residuals:
    Min      1Q  Median      3Q     Max
-17.068  -5.433   0.780   5.211  41.456

Coefficients:
```

```
               Estimate Std. Error t value Pr(>|t|)
(Intercept)     380.384      7.083  53.703   <2e-16 ***
post_treatment  -29.710      1.870 -15.886   <2e-16 ***
p2price          -4.068      2.570  -1.583    0.117
p2prom           78.256      2.125  36.831   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 8.225 on 100 degrees of freedom
Multiple R-squared:  0.9361,    Adjusted R-squared:  0.9342
F-statistic: 488.1 on 3 and 100 DF,  p-value: < 2.2e-16
```

A simple two-way fixed-effect panel model for each product controlling for relevant covariates

Store fixed effects will automatically absorb (control for) all time-invariant store characteristics (like storesize and city). Time fixed effects will absorb aggregate shocks that affect all stores in a given week. By accounting for these factors, the TWFE model addresses the primary endogeneity problem of unobserved effects that are either subject-invariant or time-invariant. While the simple pre-post model assumes the counterfactual outcome remains constant over time, the TWFE model (within a Difference-in-Differences framework) relaxes this by assuming that treatment and control groups would follow parallel trends in the absence of treatment. Where this is assumed to produce smaller standard errors and tighter confidence intervals and allowing for a more granular analysis of heterogeneity on the individual and time dimensions rather than just group-level averages in contrast to pre-post analysis, the introduction of the parallel trends assumption also means that if the treated store had a unique sales trajectory that was not shared by any control stores, the TWFE estimate could be biased.

```r
# Convert the data frame to a panel data frame for plm
pdf <- pdata.frame(df, index = c("storeNum", "Weekind"))

# --- Product 1: TWFE Model ---
twfe_p1 <- plm(p1sales ~ treated + p1price + p1prom,
               data = pdf,
               model = "within",
               effect = "twoways")
summary(twfe_p1)
```

```
Twoways effects Within Model

Call:
plm(formula = p1sales ~ treated + p1price + p1prom, data = pdf,
    effect = "twoways", model = "within")

Balanced Panel: n = 21, T = 104, N = 2184
```

```
Residuals:
      Min.    1st Qu.     Median    3rd Qu.       Max.
-51.171933  -4.976294  -0.031958   4.792511  47.537512

Coefficients:
        Estimate Std. Error t-value  Pr(>|t|)
treated 29.97912    2.56688 11.6792 < 2.2e-16 ***
p1price -2.47458    0.81541 -3.0348  0.002437 **
p1prom  37.59018    0.66334 56.6678 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Total Sum of Squares:    681580
Residual Sum of Squares: 257950
R-Squared:      0.62155
Adj. R-Squared: 0.59837
F-statistic: 1126.1 on 3 and 2057 DF, p-value: < 2.22e-16
```

```
# --- Product 2: TWFE Model ---
twfe_p2 <- plm(p2sales ~ treated + p2price + p2prom,
               data = pdf,
               model = "within",
               effect = "twoways")
summary(twfe_p2)
```

```
Twoways effects Within Model

Call:
plm(formula = p2sales ~ treated + p2price + p2prom, data = pdf,
    effect = "twoways", model = "within")

Balanced Panel: n = 21, T = 104, N = 2184

Residuals:
    Min.  1st Qu.   Median  3rd Qu.     Max.
-67.5821  -6.1317  -0.3684   5.8279  52.7506

Coefficients:
         Estimate Std. Error  t-value  Pr(>|t|)
treated -34.29122    3.07136 -11.1649 < 2.2e-16 ***
```

```
p2price  -2.95105    0.89513  -3.2968 0.0009947 ***
p2prom   89.42444    0.83716 106.8185 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Total Sum of Squares:    2424600
Residual Sum of Squares: 368880
R-Squared:      0.84786
Adj. R-Squared: 0.83854
F-statistic: 3821.11 on 3 and 2057 DF, p-value: < 2.22e-16
```

Correcting for autocorrelation and heteroskedasticity

```
# --- Product 1 TWFE with Clustered Standard Errors ---
coeftest(twfe_p1, vcov = vcovHC(twfe_p1, type = "HC1", cluster = "group"))
```

```
t test of coefficients:

        Estimate Std. Error t value  Pr(>|t|)
treated  29.9791     4.0348  7.4301 1.582e-13 ***
p1price  -2.4746     0.8446 -2.9299  0.003428 **
p1prom   37.5902     1.3932 26.9812 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# --- Product 2 TWFE with Clustered Standard Errors ---
coeftest(twfe_p2, vcov = vcovHC(twfe_p2, type = "HC1", cluster = "group"))
```

```
t test of coefficients:

         Estimate Std. Error t value  Pr(>|t|)
treated -34.29122    4.45448 -7.6981 2.129e-14 ***
p2price  -2.95105    0.81652 -3.6142 0.0003085 ***
p2prom   89.42444    4.30234 20.7851 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

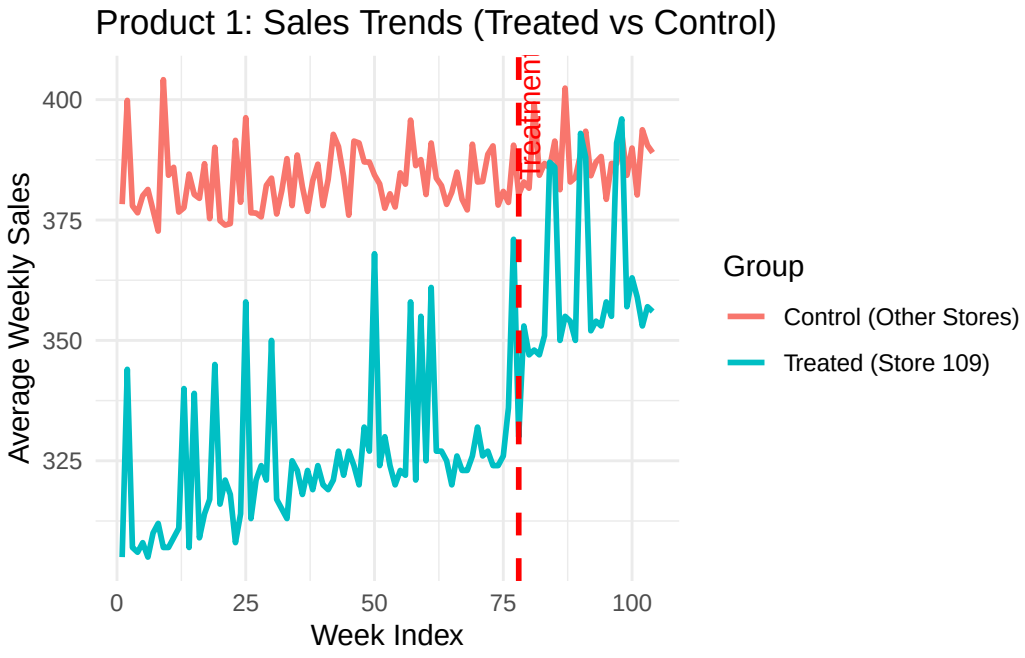
Simple DiD, SC, and SDID analyses for each product

Check for parallel trends assumption

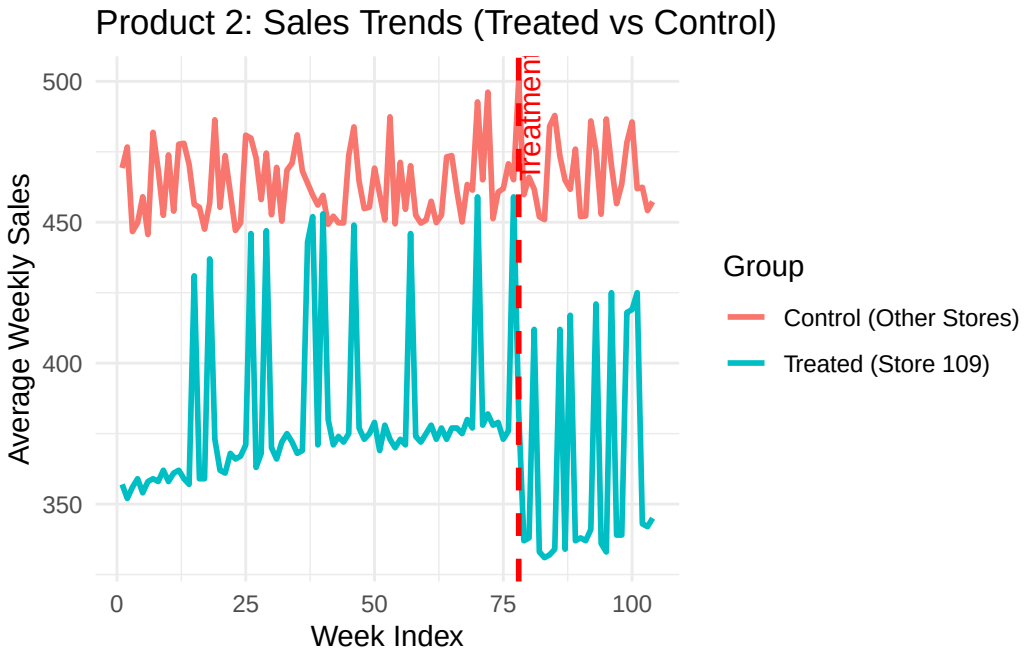
Visual inspection of pre-treatment trends

Ideally, the treated store's sales should follow a similar trajectory to the control stores in the pre-treatment period. If the treated store had a unique sales pattern that was not shared by any control stores, this would violate the parallel trends assumption and could bias our estimates. By plotting the average sales of the treated store against the average sales of the control stores over time, we can visually assess whether they exhibit similar trends before the treatment starts (Weekind < 78). If they do, this supports the validity of our DiD and SDID analyses; if not, we may need to consider alternative methods or be cautious in interpreting our results.

```
# --- Product 1 Plot ---
# Aggregate average sales per week for treated vs control stores
trend_data_p1 <- df %>%
  group_by(Weekind, is_treated_store) %>%
  summarise(mean_sales = mean(p1sales, na.rm = TRUE), .groups = 'drop') %>%
  mutate(Group = ifelse(is_treated_store == 1, "Treated (Store 109)", "Control (Other Stores)"))

plot_p1 <- ggplot(trend_data_p1, aes(x = Weekind, y = mean_sales, color = Group)) +
  geom_line(linewidth = 1) +
  geom_vline(xintercept = 78, linetype = "dashed", color = "red", linewidth = 1) +
  annotate("text", x = 76, y = max(trend_data_p1$mean_sales),
           label = "Treatment Start", angle = 90, vjust = 1.5, color = "red") +
  labs(title = "Product 1: Sales Trends (Treated vs Control)",
       x = "Week Index",
       y = "Average Weekly Sales") +
  theme_minimal()

print(plot_p1)
```



```r
# --- Product 2 Plot ---
trend_data_p2 <- df %>%
  group_by(Weekind, is_treated_store) %>%
  summarise(mean_sales = mean(p2sales, na.rm = TRUE), .groups = 'drop') %>%
  mutate(Group = ifelse(is_treated_store == 1, "Treated (Store 109)", "Control (Other Stores)"))

plot_p2 <- ggplot(trend_data_p2, aes(x = Weekind, y = mean_sales, color = Group)) +
  geom_line(linewidth = 1) +
  geom_vline(xintercept = 78, linetype = "dashed", color = "red", linewidth = 1) +
  annotate("text", x = 76, y = max(trend_data_p2$mean_sales),
           label = "Treatment Start", angle = 90, vjust = 1.5, color = "red") +
  labs(title = "Product 2: Sales Trends (Treated vs Control)",
       x = "Week Index",
       y = "Average Weekly Sales") +
  theme_minimal()

print(plot_p2)
```



Statistical test for parallel trends: Pre-treatment interaction model

Additionally to doing the visual inspection, we can also run a formal statistical test for parallel trends by regressing sales on the store dummy, a linear time trend (Weekind), and their interaction, but only using the pre-treatment data (Weeks 1 to 77). If the coefficient on the interaction term is statistically significant, it would suggest that the treated store had a different trend compared to the control stores even before the treatment started, which would violate the parallel trends assumption and potentially bias our DiD and SDID estimates.

```r
# Filter the data to ONLY include the pre-treatment period (Weeks 1 to 77)
df_pre <- df %>% filter(Weekind < 78)

# --- Product 1: Parallel Trends Test ---
pt_test_p1 <- lm(p1sales ~ is_treated_store * Weekind, data = df_pre)
summary(pt_test_p1)
```

```
Call:
lm(formula = p1sales ~ is_treated_store * Weekind, data = df_pre)

Residuals:
     Min       1Q   Median       3Q      Max
-169.496  -49.819   -0.924   46.066  168.571

Coefficients:
                          Estimate Std. Error t value Pr(>|t|)
(Intercept)              381.56235    3.47935 109.665  < 2e-16 ***
is_treated_store         -68.18266   15.94439  -4.276 2.01e-05 ***
Weekind                    0.03809    0.07751   0.491    0.623
is_treated_store:Weekind   0.24354    0.35520   0.686    0.493
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 67.61 on 1613 degrees of freedom
Multiple R-squared:  0.03364,   Adjusted R-squared:  0.03185
F-statistic: 18.72 on 3 and 1613 DF,  p-value: 6.166e-12
```

```r
# --- Product 2: Parallel Trends Test ---
pt_test_p2 <- lm(p2sales ~ is_treated_store * Weekind, data = df_pre)
summary(pt_test_p2)
```

```
Call:
lm(formula = p2sales ~ is_treated_store * Weekind, data = df_pre)

Residuals:
    Min      1Q  Median      3Q     Max
-243.39  -71.42   -8.20   76.70  333.94

Coefficients:
```

```
                           Estimate Std. Error t value Pr(>|t|)
(Intercept)              462.922368   5.666993  81.687  < 2e-16 ***
is_treated_store         -96.208766  25.969426  -3.705 0.000219 ***
Weekind                    0.006553   0.126245   0.052 0.958611
is_treated_store:Weekind   0.346778   0.578529   0.599 0.548981
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 110.1 on 1613 degrees of freedom
Multiple R-squared:  0.02523,   Adjusted R-squared:  0.02341
F-statistic: 13.91 on 3 and 1613 DF,  p-value: 5.857e-09
```

Findings: For both products, the visual interpretation leads to believe that there is a slight positive trend in the treated store's sales compared to the control stores in the pre-treatment period, which could potentially violate the parallel trends assumption. However, the formal statistical test for parallel trends (the interaction term between the treated store dummy and the linear time trend [is_treated_store:Weekind]) is not statistically significant for either product ($p > 0.05$). This suggests that while there may be some visual differences in trends, they are not statistically significant enough to conclusively reject the parallel trends assumption possibly due to limited sample size or high variability. Therefore, we can proceed with our DiD and SDID analyses, but we should be cautious in interpreting the results given the potential for some underlying differences in pre-treatment trends.

Evaluating the treatment effect using DiD, SC, and SDID methods for each product

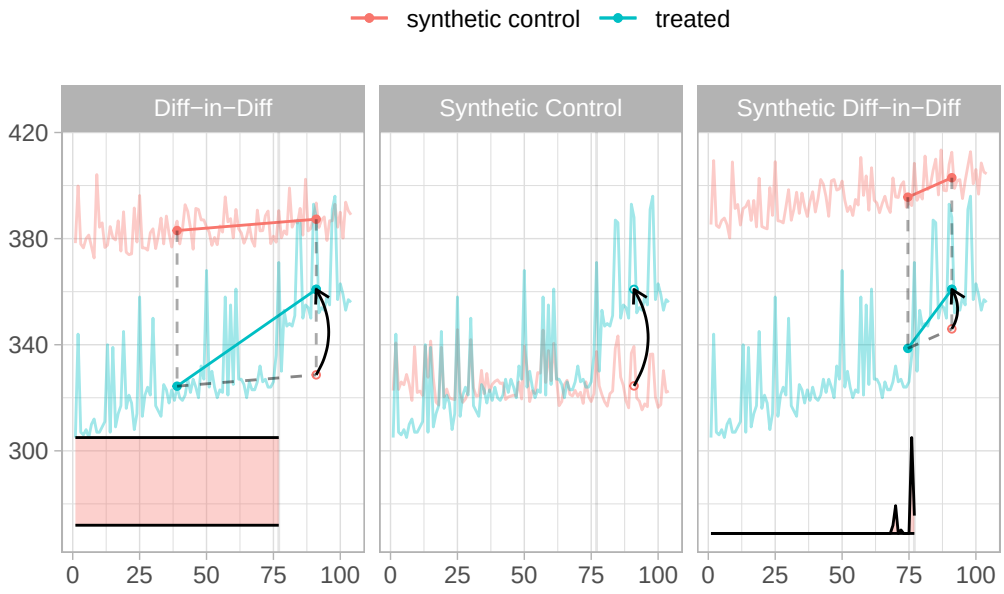
Product 1: p1sales

```
# Ensure data frame is standard class (synthdid prefers plain data.frames over tibbles)
df <- as.data.frame(df)

# Setup panel matrices for Product 1
setup_p1 <- panel.matrices(
  df,
  unit = "storeNum",
  time = "Weekind",
  outcome = "p1sales",
  treatment = "treated"
)

# Estimate the treatment effects for Product 1
sdid_est_p1 <- synthdid_estimate(setup_p1$Y, setup_p1$N0, setup_p1$T0)
did_est_p1 <- did_estimate(setup_p1$Y, setup_p1$N0, setup_p1$T0)
sc_est_p1 <- sc_estimate(setup_p1$Y, setup_p1$N0, setup_p1$T0)
estimates_p1 = list(did_est_p1, sc_est_p1, sdid_est_p1)
```

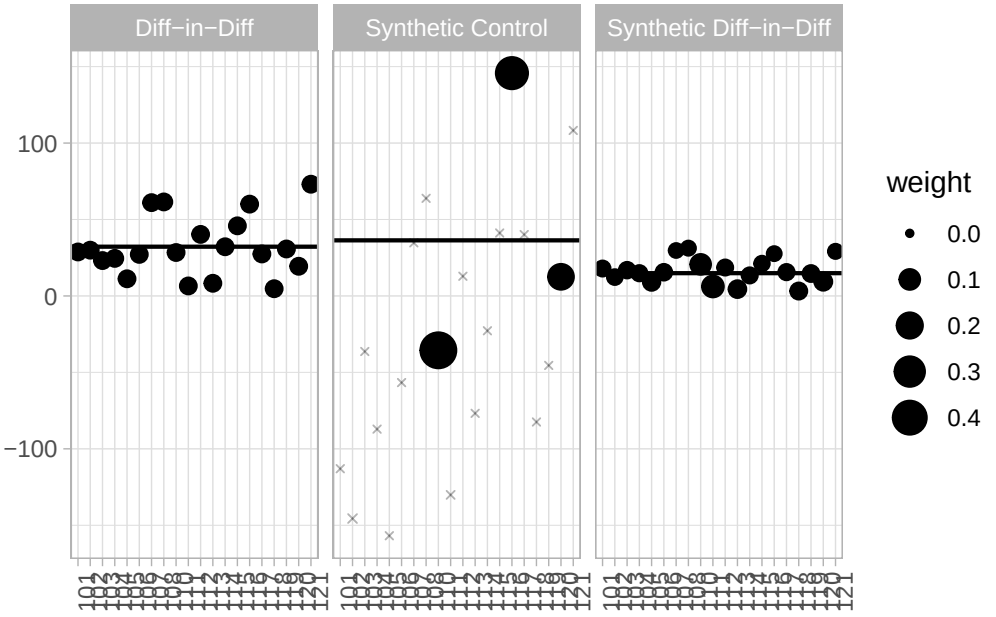
```
# Print the estimates
names(estimates_p1) = c('Diff-in-Diff', 'Synthetic Control', 'Synthetic Diff-in-Diff')
print(unlist(estimates_p1))
```

```
          Diff-in-Diff       Synthetic Control Synthetic Diff-in-Diff
              32.21775               36.37194               14.88871
```

```
# Plot 1: Trends of treated vs. synthetic control
synthdid_plot(estimates_p1, facet.vertical = FALSE)
```




```
# Plot 2: See which control stores received the highest weights
synthdid_units_plot(estimates_p1)
```



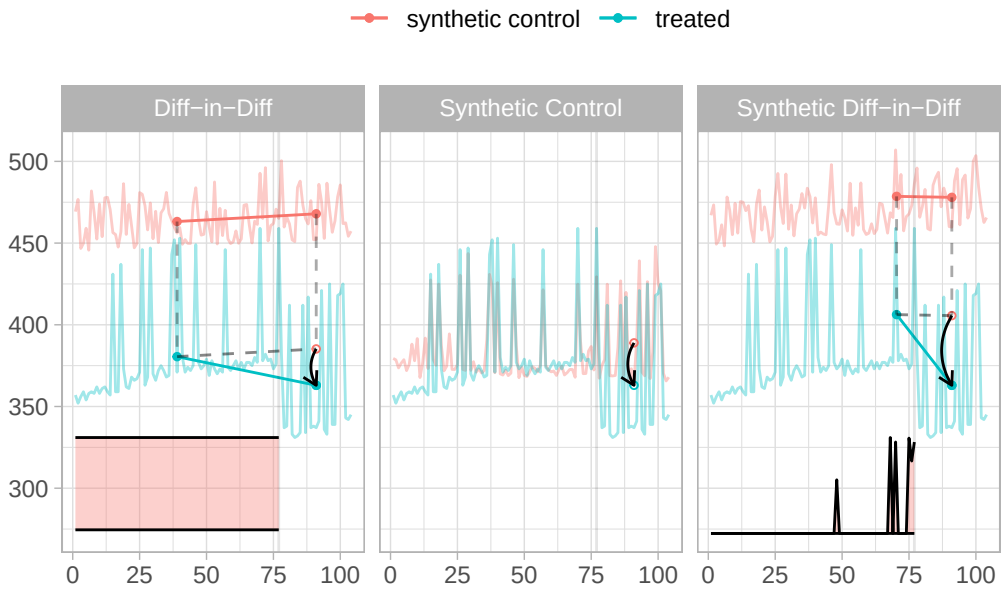
Product 2: p2sales

The same steps as for Product 1 are repeated for Product 2.

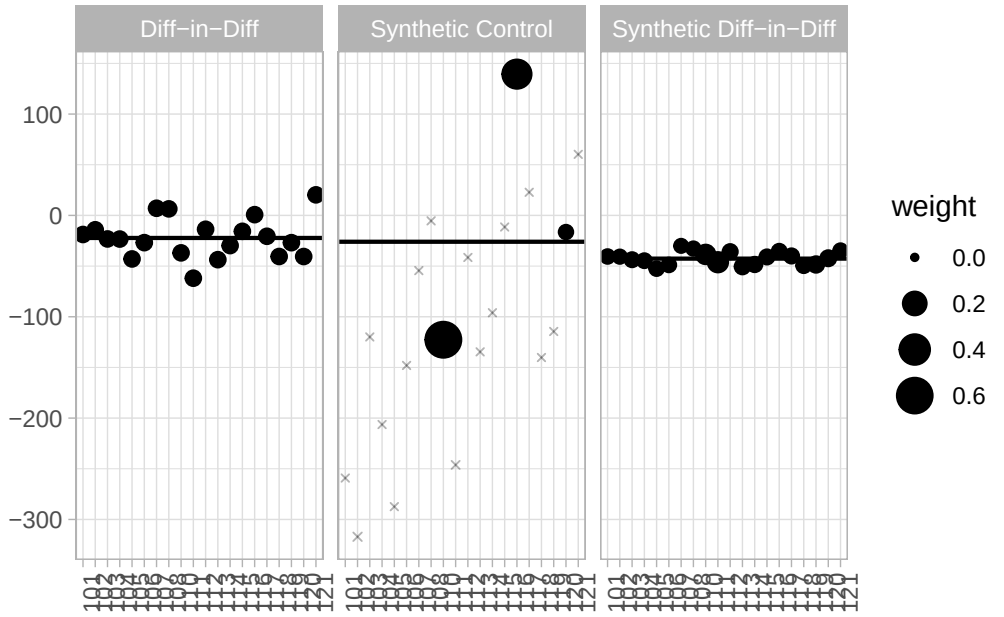
```
# Setup panel matrices for Product 2
setup_p2 <- panel.matrices(
  df,
  unit = "storeNum",
  time = "Weekind",
  outcome = "p2sales",
  treatment = "treated"
)

# Estimate the treatment effects for Product 2
sdid_est_p2 <- synthdid_estimate(setup_p2$Y, setup_p2$N0, setup_p2$T0)
did_est_p2 <- did_estimate(setup_p2$Y, setup_p2$N0, setup_p2$T0)
sc_est_p2 <- sc_estimate(setup_p2$Y, setup_p2$N0, setup_p2$T0)
estimates_p2 = list(did_est_p2, sc_est_p2, sdid_est_p2)

# Print the estimates
names(estimates_p2) = c('Diff-in-Diff', 'Synthetic Control', 'Synthetic Diff-in-Diff')
print(unlist(estimates_p2))
```

```
          Diff-in-Diff       Synthetic Control Synthetic Diff-in-Diff
             -22.25818               -25.97635               -42.64470
```

```
# Plot 1: Trends of treated vs. synthetic control
synthdid_plot(estimates_p2, facet.vertical = FALSE)
```




```
# Plot 2: See which control stores received the highest weights
synthdid_units_plot(estimates_p2)
```



Expensions to the analysis

Two possible expansions to the analysis include:

1. Running a panel regression-based difference-in-differences model that includes relevant covariates for “pooling”. This implies that it treats the entire dataset as a single pool of observations and uses Ordinary Least Squares (OLS) to estimate the coefficients for dummy veriables to handle store-level and time-period variation.
2. Implementing a version of the SDID method that first regresses out the effects of price and promotions (using the Frisch-Waugh-Lovell theorem) before applying the SDID estimator to the residuals. This will allow us to see how controlling for these covariates in a more flexible way (without assuming linearity or additivity) affects the estimates and their precision.

Panel regression difference-in-differences

To further explore the data, we can also run a panel regression with a difference-in-differences specification that includes the relevant covariates. This will allow us to compare the results from this more traditional econometric approach with the estimates obtained from the SDID method.

Product 1: Panel DiD WITH Covariates

```
did_cov_p1 <- plm(p1sales ~ is_treated_store + post_treatment + treated +
                            p1price + p1prom + compind + storesize + citysize,
                  data = pdf,
                  model = "pooling")
summary(did_cov_p1)
```

```
Pooling Model

Call:
plm(formula = p1sales ~ is_treated_store + post_treatment + treated +
    p1price + p1prom + compind + storesize + citysize, data = pdf,
    model = "pooling")

Balanced Panel: n = 21, T = 104, N = 2184

Residuals:
     Min.   1st Qu.    Median   3rd Qu.      Max.
-50.36165  -5.27904   0.36831   5.32015  44.24457

Coefficients:
                    Estimate  Std. Error  t-value  Pr(>|t|)
(Intercept)       2.0488e+02  2.1580e+00  94.9419 < 2.2e-16 ***
is_treated_store -3.3633e+01  1.3414e+00 -25.0730 < 2.2e-16 ***
post_treatment    3.5381e+00  5.5270e-01   6.4015 1.879e-10 ***
treated           2.9974e+01  2.5287e+00  11.8535 < 2.2e-16 ***
p1price          -2.5853e+00  7.7822e-01  -3.3220 0.0009085 ***
```

```
p1prom            3.7619e+01  5.9851e-01  62.8544 < 2.2e-16 ***
compind           9.3823e+01  6.8649e-01 136.6696 < 2.2e-16 ***
storesize         1.2244e+00  7.8548e-03 155.8758 < 2.2e-16 ***
citysize          1.0299e-04  1.0029e-06 102.7016 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Total Sum of Squares:    11379000
Residual Sum of Squares: 264690
R-Squared:      0.97674
Adj. R-Squared: 0.97665
F-statistic: 11416 on 8 and 2175 DF, p-value: < 2.22e-16
```

```
# Product 1: Covariate DiD with Clustered Standard Errors
coeftest(did_cov_p1, vcov = vcovHC(did_cov_p1, type = "HC1", cluster = "group"))
```

```
t test of coefficients:

                   Estimate  Std. Error  t value  Pr(>|t|)
(Intercept)       2.0488e+02  2.5912e+00  79.0685 < 2.2e-16 ***
is_treated_store -3.3633e+01  1.0830e+00 -31.0553 < 2.2e-16 ***
post_treatment    3.5381e+00  3.9689e+00   0.8915 0.3727792
treated           2.9974e+01  4.0396e+00   7.4198 1.672e-13 ***
p1price          -2.5853e+00  7.8067e-01  -3.3116 0.0009428 ***
p1prom            3.7619e+01  1.4219e+00  26.4564 < 2.2e-16 ***
compind           9.3823e+01  3.3419e-01 280.7499 < 2.2e-16 ***
storesize         1.2244e+00  2.0424e-03 599.4855 < 2.2e-16 ***
citysize          1.0299e-04  2.7410e-07 375.7586 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Product 2: Panel DiD WITH Covariates

```
did_cov_p2 <- plm(p2sales ~ is_treated_store + post_treatment + treated +
                            p2price + p2prom + compind + storesize + citysize,
                  data = pdf,
                  model = "pooling")
summary(did_cov_p2)
```

```
Pooling Model

Call:
```

```
plm(formula = p2sales ~ is_treated_store + post_treatment + treated +
    p2price + p2prom + compind + storesize + citysize, data = pdf,
    model = "pooling")

Balanced Panel: n = 21, T = 104, N = 2184

Residuals:
     Min.   1st Qu.    Median   3rd Qu.      Max.
-67.46766  -6.34606   0.14586   6.03922  53.12151

Coefficients:
                    Estimate  Std. Error t-value  Pr(>|t|)
(Intercept)       1.5259e+02  2.5743e+00  59.276 < 2.2e-16 ***
is_treated_store -1.8371e+01  1.6208e+00 -11.335 < 2.2e-16 ***
post_treatment    2.9050e+00  6.6751e-01   4.352 1.411e-05 ***
treated          -3.4354e+01  3.0600e+00 -11.227 < 2.2e-16 ***
p2price          -2.8813e+00  8.6397e-01  -3.335 0.0008673 ***
p2prom            8.9915e+01  7.7995e-01 115.284 < 2.2e-16 ***
compind           9.4394e+01  8.3034e-01 113.681 < 2.2e-16 ***
storesize         2.0648e+00  9.5007e-03 217.331 < 2.2e-16 ***
citysize          2.0847e-04  1.2128e-06 171.898 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Total Sum of Squares:    28441000
Residual Sum of Squares: 387230
R-Squared:      0.98638
Adj. R-Squared: 0.98633
F-statistic: 19696.6 on 8 and 2175 DF, p-value: < 2.22e-16
```

```
# Product 2: Covariate DiD with Clustered Standard Errors
coeftest(did_cov_p2, vcov = vcovHC(did_cov_p2, type = "HC1", cluster = "group"))
```

```
t test of coefficients:

                    Estimate  Std. Error  t value  Pr(>|t|)
(Intercept)       1.5259e+02  2.3256e+00  65.6149 < 2.2e-16 ***
is_treated_store -1.8371e+01  1.1091e+00 -16.5636 < 2.2e-16 ***
post_treatment    2.9050e+00  4.0676e+00   0.7142    0.4752
```

```
treated          -3.4354e+01  4.4606e+00  -7.7016 2.025e-14 ***
p2price          -2.8813e+00  7.2146e-01  -3.9938 6.719e-05 ***
p2prom            8.9915e+01  4.7046e+00  19.1123 < 2.2e-16 ***
compind           9.4394e+01  1.9037e-01 495.8359 < 2.2e-16 ***
storesize         2.0648e+00  3.7319e-03 553.2849 < 2.2e-16 ***
citysize          2.0847e-04  3.5877e-07 581.0723 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Synthetic DiD regressing out the covariates

Given that the base synthdid package used does not easily accept additional control variables (like price and promotions) directly into its primary matrix setup, we can implement a solution based on the Frisch-Waugh-Lovell (FWL) theorem: we first use a linear regression to predict sales based only on the covariates, extract the residuals (the portion of sales unexplained by price, promotions, etc.), and then apply the SDID model to those residuals.

By adding the overall mean back to the residuals, we keep the interpretation on the same scale as the original sales volume.

Product 1: p1sales

```
# Run a linear model and extract the residuals (unexplained sales)
model_p1 <- lm(p1sales ~ p1price + p1prom + compind + storesize + citysize, data = df)
df$p1sales_adj <- residuals(model_p1) + mean(df$p1sales, na.rm = TRUE)

# Setup panel matrices using the NEW adjusted outcome variable
setup_p1_adj <- panel.matrices(
  df,
  unit = "storeNum",
  time = "Weekind",
  outcome = "p1sales_adj",
  treatment = "treated"
)

# Estimate the adjusted SDID Treatment Effect
sdid_est_p1_adj <- synthdid_estimate(setup_p1_adj$Y, setup_p1_adj$N0, setup_p1_adj$T0)
print(sdid_est_p1_adj)
```

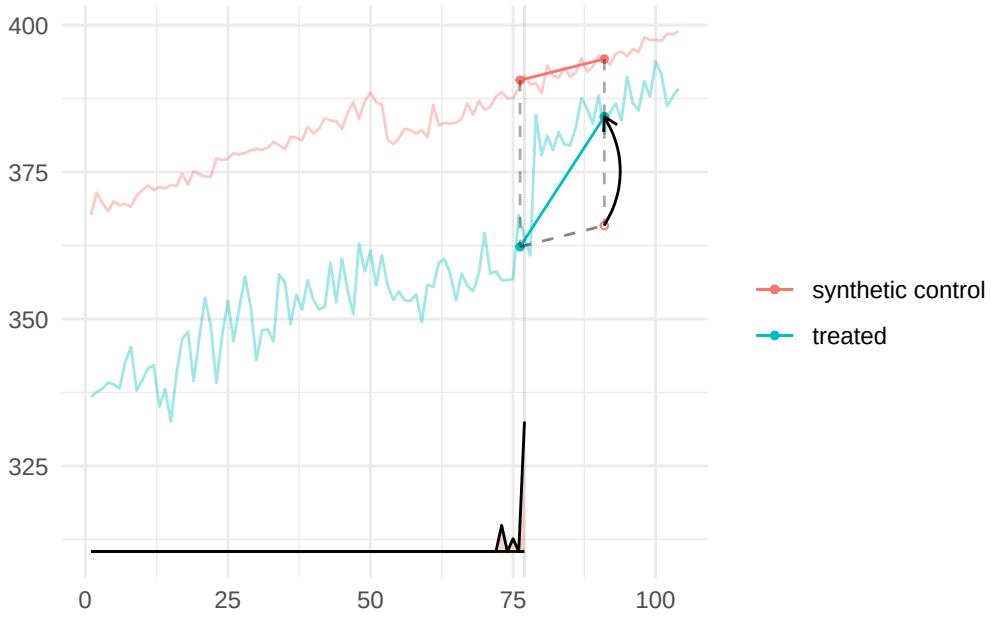
```
synthdid: 18.519 +- NA. Effective N0/N0 = 14.1/20~0.7.
Effective T0/T0 = 1.6/77~0.0. N1,T1 = 1,27.
```

```
# Calculate and print confidence intervals for Product 1
se_p1_adj <- sqrt(vcov(sdid_est_p1_adj, method = "placebo"))
cat(sprintf("Adjusted 95%% CI: [%.2f, %.2f]\n\n",
            sdid_est_p1_adj - 1.96 * se_p1_adj,
```

```
            sdid_est_p1_adj + 1.96 * se_p1_adj))
```

```
Adjusted 95% CI: [11.78, 25.26]
```

```
# Visualize adjusted trends
plot(sdid_est_p1_adj)+theme_minimal()
```



Product 2: p2sales

```
# Run a linear model and extract the residuals (unexplained sales)
model_p2 <- lm(p2sales ~ p2price + p2prom + compind + storesize + citysize, data = df)
df$p2sales_adj <- residuals(model_p2) + mean(df$p2sales, na.rm = TRUE)

# Setup panel matrices using the NEW adjusted outcome variable
setup_p2_adj <- panel.matrices(
  df,
  unit = "storeNum",
  time = "Weekind",
  outcome = "p2sales_adj",
  treatment = "treated"
)

# Estimate the adjusted SDID Treatment Effect
sdid_est_p2_adj <- synthdid_estimate(setup_p2_adj$Y, setup_p2_adj$N0, setup_p2_adj$T0)
print(sdid_est_p2_adj)
```

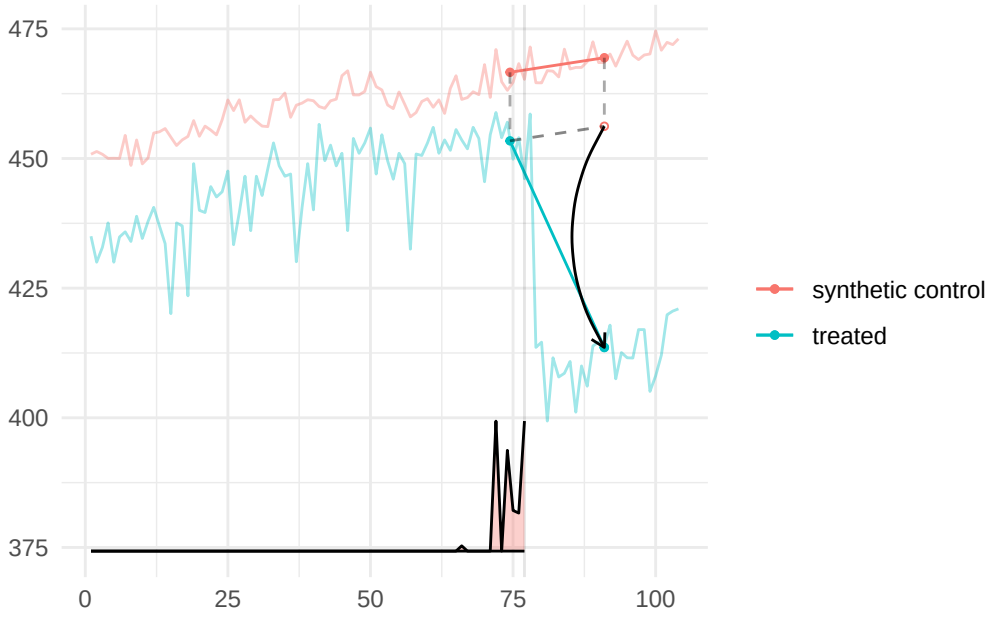
```
synthdid: -42.649 +- NA. Effective N0/N0 = 16.2/20~0.8.
```

```
Effective T0/T0 = 4.2/77~0.1. N1,T1 = 1,27.
```

```
# Calculate and print confidence intervals for Product 2
se_p2_adj <- sqrt(vcov(sdid_est_p2_adj, method = "placebo"))
cat(sprintf("Adjusted 95%% CI: [%.2f, %.2f]\n\n",
            sdid_est_p2_adj - 1.96 * se_p2_adj,
            sdid_est_p2_adj + 1.96 * se_p2_adj))
```

```
Adjusted 95% CI: [-56.22, -29.08]
```

```
# Visualize adjusted trends
plot(sdid_est_p2_adj)+theme_minimal()
```



Summary of Findings

Table 3

Comparison of Treatment Effect Estimates for Product 1 and Product 2

Model	P1 Estimate	P1 SE	P2 Estimate	P2 SE
Pre-Post (LM)	33.69	1.71	-29.71	1.87
Two-Way Fixed Effects (TWFE)	29.98	4.03	-34.29	4.45
Difference-in-Differences (DID)	32.22	19.93	-22.26	20.81
Panel DID (with Covariates)	29.97	4.04	-34.35	4.46
Synthetic Control (SC)	36.37	13.55	-25.98	31.58
Synthetic DID (SDID)	14.89	8.62	-42.64	10.58
Adjusted SDID (with Covariates)	18.52	3.63	-42.65	6.55

Conclusions

Pre-Post Analysis

In the Pre-Post analysis of Store 109, the pre-post model yielded the following estimates:

- **Product 1:** A significant increase of approximately 33.69 units $(p < 0.001)$
- **Product 2:** A significant decrease of approximately 29.71 units $(p < 0.001)$

While these numbers provide a clear description of the sales change at Store 109, they are only "appropriate" causal estimates if no other external factors influenced sales between the pre- and post-treatment weeks. If, for example, a competitor ran a major promotion on Product 2 during the same week as the treatment, the pre-post model would wrongly conclude that the in-store marketing caused the sales drop. Therefore, while useful for initial descriptive analysis, more advanced methods like the Two-Way Fixed Effect (TWFE) model or Synthetic Control are generally more appropriate for establishing causality in this dataset as they account for time-varying aggregate shocks.

Two-Way Fixed Effects (TWFE) Analysis

In the Two-Way Fixed Effects analysis of Store 109, the TWFE model provided more nuanced estimates compared to the naive Pre-Post model:

- **Product 1:** The TWFE estimate was 29.98 units (SE: 4.03), slightly lower and more conservative than the Pre-Post estimate of 33.69
- **Product 2:** The TWFE estimate was -34.29 units (SE: 4.45), showing a more pronounced decrease than the Pre-Post estimate of -29.71

These differences highlight that the TWFE model is more appropriate than the Pre-Post model because it adjusts for weekly sales fluctuations across the entire retail chain, preventing those general trends from being incorrectly attributed to the specific in-store marketing treatment.

Difference-in-Differences (DID) Analysis

In the Difference-in-Differences analysis of Store 109, the simple DID model provided the following results:

- **Product 1:** Estimate of 32.22 units (SE: 19.88)
- **Product 2:** Estimate of -22.26 units (SE: 20.14)

Notably, the standard errors for the simple DID in this specific task were significantly larger than those of the Two-Way Fixed Effects (TWFE) or Adjusted SDID models. This suggests that while a simple DID is a theoretical improvement over a pre-post model, it may lack precision in datasets with high variance between stores unless more granular controls (like fixed effects or matching) are incorporated. While these additions could possibly reduce the standard errors, by looking at the graph, we can see that the data does not follow the parallel trends assumption very well, which is likely contributing to the large standard errors. Therefore, while the simple DID is a step in the right direction for causal inference, it may not be the most appropriate method for this dataset without further adjustments.

Panel DiD with Covariates Analysis

In the Panel DiD with Covariates analysis of Store 109, the model provided the following estimates:

- **Product 1:** Estimate of 29.97 units (SE: 4.04)
- **Product 2:** Estimate of -34.35 units (SE: 4.46)

The estimates for Product 1 (29.98 vs 29.97) and Product 2 (-34.29 vs -34.35) are almost identical to the TWFE model's estimates. This indicates that the explicit covariates included in the Panel DID model successfully captured nearly all the systematic store-level differences that the TWFE model's fixed effects would have absorbed. Also like the TWFE model, the Panel DiD with Covariates model shows very high precision (SEs ~4.0) compared to a Simple DID (SEs ~20.0). This is posibly because both models include prices and promotions as covariates, which tend to have noticeable impact on sales volume. By explaining this variance, both models significantly reduce the noise in the residuals. With that said, the Simple TWFE model is generally considered more robust because it controls for all store characteristics, even those that are not seen or measured. If there were an unobserved store difference not captured by *citysize* or *storesize*, the pooling model could be biased. However, the near-identical results suggest that these covariates were comprehensive for this specific task.

Synthetic Control (SC) Analysis

In the Synthetic Control analysis of Store 109, the SC model provided the following estimates:

- **Product 1:** Estimate of 36.37 units (SE: 11.83)
- **Product 2:** Estimate of -25.98 units (SE: 33.54)

While SC provided the highest positive estimate for Product 1, its standard errors in this specific task were notably larger than those of more advanced hybrid methods like Synthetic DID (SDID) or Two-Way Fixed Effects (TWFE). This also aligns with the visual inspection of the trends, where the synthetic control had a reasonable pre-treatment fit but due to the high amount of noise in the data, the post-treatment estimate is less precise and potentially overfitted. This suggests that while SC is appropriate for defining the counterfactual based on pre-treatment trends, its precision can be lower in retail panels unless combined with additional regularization or time-weighting (as seen in SDID).

Synthetic Difference-in-Differences (SDID) and Adjusted SDID Analysis

In the Synthetic Difference-in-Differences analysis of Store 109, the SDID models provided the following estimates:

- **Product 1:** Pure SDID Estimate of 14.89 (SE: 8.63)
- **Product 2:** Pure SDID Estimate of -42.64 (SE: 10.14)
- **Product 1:** Adjusted SDID Estimate of 18.52 (SE: 3.64)
- **Product 2:** Adjusted SDID Estimate of -42.65 (SE: 6.80)

Precision: The Adjusted SDID yielded the smallest standard errors for both products. This was achieved by using the Frisch-Waugh-Lovell (FWL) theorem to first "regress out" the effects of prices and

promotions before applying the SDID estimator.

Conservatism vs. Magnitude: For Product 1, the SDID estimate (18.52) was significantly more conservative than the naive Pre-Post (33.69) or simple DID (32.22) models. This suggests that earlier models were likely overestimating the marketing effect by failing to account for specific control units that shared Store 109's unique sales trajectory.

Reliability: For Product 2, the SDID estimate (-42.65) showed a much stronger negative impact than the simple DID (-22.26). This indicates that when control units are reweighted to match the treated store's specific behavior, the sales drop appears even more severe.

Final considerations: The SDID approach, especially when adjusted for covariates, appears to provide the most reliable and precise estimates of the treatment effect in this retail panel context. It effectively combines the strengths of both SC (defining a data-driven counterfactual) and DID (leveraging time variation), while also controlling for confounding factors through regression adjustment. However it is very important to consider: because SDID often weights only a few time periods directly before the treatment, the estimate can be sensitive to noise in those specific weeks; to prevent over-reliance on a single control unit and ensure a unique solution, SDID uses a regularization constant (ξ) to disperse unit weights; similar to SC, the credibility of SDID rests on the quality of the pre-treatment fit; if the weighted controls cannot reasonably mimic the treated unit's trend, the resulting estimate may be biased. With those considerations in mind, the SDID method, particularly when adjusted for covariates, appears to be the most appropriate and reliable method for estimating the causal effect of the in-store marketing strategy on sales in this dataset.

Final decision:

Recommendations to Management

The primary recommendation is to proceed with caution regarding a full-scale rollout of this marketing strategy. While the strategy successfully drives sales for Product 1, it appears to have a significant negative impact on Product 2 sales. Management should investigate whether this represents a "cannibalization" effect or if the marketing for Product 1 is actively deterring Product 2 purchases. Before a chain-wide implementation, it is essential to calculate the net profit margin of these changes, as the volume loss in Product 2 (~42 units) is more than double the volume gain in Product 1 (~18 units)

Effectiveness and Expected Sales Uplift

Based on the most precise and realiable model [*Adjusted Synthetic DID (SDID)*] the strategy is effective at changing sales levels, but the direction varies by product:

- **Product 1:** You can expect a statistically significant expected uplift of approximately 18.52 units per week (SE: 3.64). This estimate is more reliable and conservative than the naive pre-post estimate (33.69), which likely overestimated the effect by failing to account for general market trends.
- **Product 2:** You should expect a statistically significant decrease of approximately 42.65 units per week (SE: 6.80). This negative impact is substantial and was consistent across all advanced models

(TWFE and SDID).

Potential Challenges in the Data

Several characteristics of the retail dataset pose challenges to established causal inference:

- **Violation of Parallel Trends:** Visual inspection and the high standard errors in the simple DID model (~20.0) suggest that Store 109 did not follow the same pre-treatment trajectory as the average of the control stores. Relying on simple models would therefore lead to biased estimates.
- **Independent Price Setting:** Because store managers set prices independently at each store, price becomes an endogenous factor that varies over both time and units. If these price changes coincide with the marketing treatment, they can confound the results.
- **Selection Bias:** The treatment was not randomized; Store 109 was specifically chosen. This introduces "selection on unobservables," where unique characteristics of that store (e.g., local management culture) might make it more or less responsive to marketing than other stores.

Ideas to Overcome These Challenges

To address the data challenges identified, the following methodological approaches were implemented:

- **Weighted Counterfactuals (SDID):** To overcome the lack of parallel trends, Synthetic DID was used, which reweights control stores and pre-treatment weeks to create a custom "synthetic" match that specifically mimics Store 109's unique trajectory.
- **Frisch-Waugh-Lovell (FWL) Theorem:** The impact of prices and promotions were addressed by "regressing them out" before performing the causal estimation. This isolation of the marketing effect significantly improved precision, reducing standard errors for Product 1 from 18.50 (Simple DID) to 3.47 (Adjusted SDID).
- **Two-Way Fixed Effects:** By including store and time fixed effects, we controlled for all stable store characteristics and national-level shocks (like holidays), solving the endogeneity problem for any factors that are either subject-invariant or time-invariant.

References

Braun, M., & Schwartz, E. M. (2025). Where A/B Testing Goes Wrong: How Divergent Delivery Affects What Online Experiments Cannot (and Can) Tell You About How Customers Respond to Advertising. *Journal of Marketing*, *89*(2), 71–95. https://doi.org/10.1177/00222429241275886

Liu, Y. (2007). The Long-Term Impact of Loyalty Programs on Consumer Purchase Behavior and Loyalty. *Journal of Marketing*, *71*(4), 19–35. https://doi.org/10.1509/jmkg.71.4.019

Stankovic, S. (2021, October). Player motivations: The carrot, the stick, and the holy grail. Retrieved February 24, 2026, from https://medium.com/ironsource-levelup/the-carrot-the-stick-and-the-holy-grail-d783f46cb7d7